LIMITBREAK · SECURITY REVIEW

# LimitBreak AMM Review.

A security review of AMM Review.

| CLIENT | PROJECT | AUDITORS | AUDIT WINDOW | FINAL REPORT |
|---|---|---|---|---|
| LimitBreak | AMM Review | Guardian | September 9 – November 12, 2025 | November 19, 2025 |

§ 00

# Contents.

134 pages — overview, confidence ranking, scope, 83 findings, disclaimer, and about Guardian.


§ 01 **Overview** P. 03

§ 02 **Confidence Ranking** P. 04

§ 03 **Scope & Classifications** P. 05

Files in Scope P. 06

§ 04 **Findings** P. 09

Main Review · 64 findings P. 15

Remediation Review · 19 findings P. 106

§ 05 **Disclaimer** P. 133

§ 06 **About Guardian** P. 134

§ 01

# Overview.

Guardian reviewed AMM Review across 2 review rounds. The final report includes 11 Critical, 15 High, 20 Medium, 16 Low, 21 Informational findings.

AUDIT FIRM
Guardian

CLIENT
LimitBreak

PROJECT
AMM Review

PREPARED BY
Guardian

AUDIT WINDOW
September 9 – November 12, 2025

FINAL REPORT
November 19, 2025

REPOSITORY
`limit-break-inc/lbamm-core`, `limit-break-inc/lbamm-hooks-and-handlers`, `limit-break-inc/lbamm-pool-type-fixed`, `limit-break-inc/lbamm-pool-type-single-provider`, `limit-break-inc/amm-pool-type-dynamic`, `limit-break-inc/tm-core-lib`

METHODOLOGY
Smart contract · Manual · Main Review · Remediation Review

MAIN REVIEW COMMIT
`f14e781f817b`

REMEDIATION REVIEW COMMIT
`b120194eca8a`

## 1.1 Confidence Ranking

**1 / 5**

**Very Low Confidence**

Guardian assigns a Confidence Ranking of 1 out of 5, combining finding severity, remediation outcome, and the reviewer's assessment of latent risk. The full scale is on the next page.

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| VERY LOW CONFIDENCE | LOW CONFIDENCE | MODERATE CONFIDENCE | HIGH CONFIDENCE | VERY HIGH CONFIDENCE |

## 1.2 Findings at a Glance

| SEVERITY | TOTAL | PENDING | ACK | PARTIAL | RESOLVED |
|---|---|---|---|---|---|
| Critical | 11 | 0 | 0 | 0 | 11 |
| High | 15 | 0 | 0 | 1 | 14 |
| Medium | 20 | 0 | 7 | 1 | 12 |
| Low | 16 | 0 | 4 | 0 | 12 |
| Informational | 21 | 0 | 6 | 0 | 15 |
| All severities | 83 | 0 | 17 | 2 | 64 |

§ 02

# Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

| CONFIDENCE RANKING | DEFINITION AND RECOMMENDATION | RISK PROFILE |
|---|---|---|
| **5 Very High Confidence** | Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit. | 0 High/Critical findings and few Low/Medium severity findings. |
| **4 High Confidence** | Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes. | 0-1 High/Critical findings per engagement week and little to no Medium severity findings. |
| **3 Moderate Confidence** | Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment. | Some High/Critical findings, with various Medium severity findings. |
| **2 Low Confidence** | Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit. | Over 6 High and Critical findings across the engagement. |
| **1 Very Low Confidence** ASSIGNED | The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review. | Over 12 High and Critical findings across the engagement. |

§ 03

# Scope & Classifications.

Guardian reviewed AMM Review in `limit-break-inc/lbamm-core, limit-break-inc/lbamm-hooks-and-handlers, limit-break-inc/lbamm-pool-type-fixed, limit-break-inc/lbamm-pool-type-single-provider, limit-break-inc/amm-pool-type-dynamic, limit-break-inc/tm-core-lib`. The scope includes 7,829 source lines across 82 files.

## 3.1 Severity Matrix

| | IMP · HIGH | IMP · MED | IMP · LOW |
|---|---|---|---|
| LIK · HIGH | Critical | High | Medium |
| LIK · MED | High | Medium | Low |
| LIK · LOW | Medium | Low | Low |

| | |
|---|---|
| IMPACT · HIGH | Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted. |
| IMPACT · MEDIUM | Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt. |
| IMPACT · LOW | Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity. |
| LIKELIHOOD · HIGH | Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained. |
| LIKELIHOOD · MEDIUM | Possible only in uncommon cases or requires a large amount of capital relative to disruption caused. |
| LIKELIHOOD · LOW | Unlikely to ever occur in production. |

# Files in Scope 1/3.

| FILE | NSLOC | TOTAL |
|---|---|---|
| src/Constants.sol | 61 | 104 |
| src/DataTypes.sol | 154 | 409 |
| src/Errors.sol | 50 | 149 |
| src/LimitBreakAMM.sol | 183 | 827 |
| src/modules/AMMModule.sol | 1,635 | 3,083 |
| src/modules/ModuleAdmin.sol | 97 | 289 |
| src/modules/ModuleFeeCollection.sol | 31 | 122 |
| src/modules/ModuleLiquidity.sol | 32 | 217 |
| src/libraries/FeeHelper.sol | 81 | 199 |
| src/libraries/LBAMMStorage.sol | 10 | 34 |
| src/libraries/PoolDecoder.sol | 12 | 46 |
| src/interfaces/ILimitBreakAMM.sol | 15 | 23 |
| src/interfaces/ILimitBreakAMMFlashloanCallback.sol | 4 | 27 |
| src/interfaces/ILimitBreakAMMPoolType.sol | 5 | 30 |
| src/interfaces/ILimitBreakAMMTransferHandler.sol | 5 | 33 |
| src/interfaces/hooks/ILimitBreakAMMLiquidityHook.sol | 5 | 31 |
| src/interfaces/hooks/ILimitBreakAMMPoolHook.sol | 5 | 24 |
| src/interfaces/hooks/ILimitBreakAMMTokenHook.sol | 5 | 29 |
| src/interfaces/core/ILimitBreakAMMEvents.sol | 68 | 108 |
| src/interfaces/core/ILimitBreakAMMFees.sol | 4 | 28 |
| src/interfaces/core/ILimitBreakAMMFlashloan.sol | 4 | 17 |
| src/interfaces/core/ILimitBreakAMMLiquidity.sol | 4 | 41 |
| src/interfaces/core/ILimitBreakAMMProtocol.sol | 4 | 17 |
| src/interfaces/core/ILimitBreakAMMSwap.sol | 4 | 56 |
| src/interfaces/core/ILimitBreakAMMTokenSettings.sol | 4 | 32 |
| src/hooks/AMMStandardHook.sol | 282 | 722 |
| src/hooks/CreatorHookSettingsRegistry.sol | 279 | 874 |
| src/hooks/DataTypes.sol | 27 | 66 |
| src/hooks/Errors.sol | 18 | 53 |
| src/hooks/interfaces/IAMMStandardHook.sol | 39 | 80 |
| src/hooks/interfaces/ICreatorHookSettingsRegistry.sol | 38 | 81 |
| src/handlers/permit/Constants.sol | 12 | 35 |
| src/handlers/permit/DataTypes.sol | 28 | 62 |
| src/handlers/permit/Errors.sol | 10 | 29 |
| src/handlers/permit/PermitTransferHandler.sol | 240 | 419 |
| src/handlers/interfaces/ITransferHandlerExecutorValidation.sol | 3 | 25 |

# Files in Scope 2/3.

| FILE | NSLOC | TOTAL |
|---|---|---|
| src/handlers/clob/CLOBTransferHandler.sol | 315 | 637 |
| src/handlers/clob/Constants.sol | 4 | 11 |
| src/handlers/clob/DataTypes.sol | 42 | 101 |
| src/handlers/clob/Errors.sol | 19 | 56 |
| src/handlers/clob/libraries/CLOBHelper.sol | 178 | 284 |
| src/handlers/clob/interfaces/ICLOBHook.sol | 4 | 25 |
| src/Constants.sol | 15 | 44 |
| src/DataTypes.sol | 92 | 223 |
| src/Errors.sol | 9 | 26 |
| src/FixedPoolType.sol | 165 | 420 |
| src/libraries/FixedHelper.sol | 815 | 1,194 |
| src/libraries/FixedPoolDecoder.sol | 19 | 54 |
| src/interfaces/IFixedPoolType.sol | 19 | 40 |
| src/Constants.sol | 8 | 23 |
| src/DataTypes.sol | 22 | 57 |
| src/Errors.sol | 6 | 17 |
| src/SingleProviderPoolType.sol | 158 | 376 |
| src/libraries/SingleProviderHelper.sol | 87 | 175 |
| src/interfaces/ISingleProviderPoolHook.sol | 11 | 30 |
| src/interfaces/ISingleProviderPoolType.sol | 10 | 19 |
| src/Constants.sol | 16 | 49 |
| src/DataTypes.sol | 76 | 192 |
| src/DynamicPoolType.sol | 295 | 609 |
| src/Errors.sol | 17 | 50 |
| src/interfaces/IDynamicPoolType.sol | 25 | 49 |
| src/libraries/BitMath.sol | 32 | 67 |
| src/libraries/DynamicHelper.sol | 318 | 669 |
| src/libraries/DynamicPoolDecoder.sol | 10 | 44 |
| src/libraries/LiquidityMath.sol | 13 | 44 |
| src/libraries/SqrtPriceMath.sol | 178 | 411 |
| src/libraries/SwapMath.sol | 73 | 143 |
| src/libraries/TickMath.sol | 146 | 237 |
| src/token/erc20/IERC20.sol | 16 | 35 |
| src/utils/structs/EnumerableSet.sol | 123 | 374 |
| src/utils/security/RoleSetClient.sol | 10 | 15 |
| src/utils/security/TstorishReentrancyGuardWithFlags.sol | 45 | 91 |

# Files in Scope 3/3.

| FILE | NSLOC | TOTAL |
|---|---|---|
| src/utils/math/FullMath.sol | 61 | 155 |
| src/utils/math/UnsafeMath.sol | 17 | 42 |
| src/utils/cryptography/EIP712.sol | 29 | 37 |
| src/utils/cryptography/EfficientHash.sol | 410 | 1,005 |
| src/utils/cryptography/Signatures.sol | 162 | 308 |
| src/utils/access/LibOwnership.sol | 79 | 169 |
| src/utils/misc/DelegateCall.sol | 65 | 130 |
| src/utils/misc/SafeCast.sol | 28 | 65 |
| src/utils/misc/StaticDelegateCall.sol | 33 | 67 |
| src/token/erc20/utils/SafeERC20.sol | 96 | 142 |

§ 04

# Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.


| ID | TITLE | CATEGORY | SEVERITY | STATUS | PAGE |
|---|---|---|---|---|---|
| **MAIN REVIEW** | | | | | |
| C-01 | Infinite refund vulnerability | Logical Error | CRITICAL | RESOLVED | p. 15 |
| C-02 | closeOrder Fails to Update inputAmountRemaining | Logical Error | CRITICAL | RESOLVED | p. 17 |
| C-03 | Reserves can be inflated without actual deposit | Logical Error | CRITICAL | RESOLVED | p. 19 |
| C-04 | Executor spoofing causes users tokens draining | Access Control | CRITICAL | RESOLVED | p. 21 |
| C-05 | Fee Growth Misaccounting in Fixed Pools | Logical Error | CRITICAL | RESOLVED | p. 23 |
| C-06 | Filled Orders Remain Cancelable in CLOB | Logical Error | CRITICAL | RESOLVED | p. 25 |
| C-07 | Head close corrupts global currentPrice | DoS | CRITICAL | RESOLVED | p. 27 |
| C-08 | Fixed Pool Height Pointer DOS | DoS | CRITICAL | RESOLVED | p. 29 |
| H-01 | Incorrect Linked List Update in `_traverseCLOB` | Logical Error | HIGH | RESOLVED | p. 31 |
| H-02 | Price shifts cause unbounded deposits | Validation | HIGH | RESOLVED | p. 33 |
| H-04 | Mapping collision merges distinct balances | Logical Error | HIGH | RESOLVED | p. 35 |
| H-05 | Partial Fills Invalidate Limit Validation | Logical Error | HIGH | PARTIALLY RESOLVED | p. 36 |
| H-06 | Wrong Function Used in _updateFixedPoolHeights | Logical Error | HIGH | RESOLVED | p. 37 |

# Findings continued.


| ID | TITLE | CATEGORY | SEVERITY | STATUS | PAGE |
|---|---|---|---|---|---|
| H-07 | Position Side Not Cleared For Empty Ranges | Logical Error | HIGH | RESOLVED | p. 38 |
| H-08 | addLiquidity Frontrun via Height Manipulation | Frontrunning | HIGH | RESOLVED | p. 39 |
| H-09 | Token hook fees can be totally bypassed | Validation | HIGH | RESOLVED | p. 40 |
| H-10 | 16-bit overflow in the protocol fee denominator | Logical Error | HIGH | RESOLVED | p. 42 |
| H-11 | Predictable IDs enable whitelist hijacking | Validation | HIGH | RESOLVED | p. 44 |
| H-12 | Uninitialized tokens allow whitelist bypass | Validation | HIGH | RESOLVED | p. 46 |
| H-13 | Tick crossing without reaching boundary | Logical Error | HIGH | RESOLVED | p. 48 |
| M-01 | The pair-token whitelist is enforced wrongly | Logical Error | MEDIUM | RESOLVED | p. 50 |
| M-02 | Users Pay Full Fee On ExactInput Partial Fills | Unexpected Behavior | MEDIUM | RESOLVED | p. 52 |
| M-03 | Minimum Protocol Fee May Not Always Be Met | Unexpected Behavior | MEDIUM | PARTIALLY RESOLVED | p. 53 |
| M-04 | addLiquidity can get stuck in an infinite loop | DoS | MEDIUM | ACKNOWLEDGED | p. 54 |
| M-05 | updateFixedPoolHeight Infinite Loop Due Rounding | Rounding | MEDIUM | RESOLVED | p. 56 |
| M-06 | Rounding Drift Causes Infinite Loop During Swaps | Rounding | MEDIUM | RESOLVED | p. 58 |
| M-07 | Double floor enables zero input | Rounding | MEDIUM | RESOLVED | p. 60 |
| M-09 | Asymmetry In FixedPool Encourages JIT Liquidity | Gaming | MEDIUM | ACKNOWLEDGED | p. 62 |
| M-10 | Pool creation bypasses initial price validation | Validation | MEDIUM | RESOLVED | p. 64 |

# Findings continued.


| ID | TITLE | CATEGORY | SEVERITY | STATUS | PAGE |
|---|---|---|---|---|---|
| M–11 | Pause bypass for afterSwap only setup | Validation | MEDIUM | RESOLVED | p. 65 |
| M–12 | Bypass of LP Whitelist via EIP-7702 | Access Control | MEDIUM | ACKNOWLEDGED | p. 66 |
| M–13 | Pricing Bounds Logic Allows Swaps Outside Limits | Logical Error | MEDIUM | ACKNOWLEDGED | p. 67 |
| M–14 | LPs Fee undercharged on exact output | Math | MEDIUM | ACKNOWLEDGED | p. 68 |
| M–15 | Fee growth miscalculation triggers underflow | Logical Error | MEDIUM | RESOLVED | p. 70 |
| M–16 | Depth wiped during rounding causes underflow | DoS | MEDIUM | RESOLVED | p. 72 |
| M–17 | Dust orders possible via scale overflow | Validation | MEDIUM | RESOLVED | p. 74 |
| M–18 | Flashloan Fee Bypass | Logical Error | MEDIUM | ACKNOWLEDGED | p. 75 |
| L–01 | transferFrom Expecting A Boolean Return Is Used | Warning | LOW | RESOLVED | p. 76 |
| L–02 | Unexpected Partial Fill Reverts | Error | LOW | RESOLVED | p. 77 |
| L–03 | WETH Input And msg.value Blocked in directSwap | Unexpected Behavior | LOW | RESOLVED | p. 78 |
| L–04 | Single Sided Liquidity Griefing | Logical Error | LOW | RESOLVED | p. 79 |
| L–05 | Height Spacing Cap Not Scaled By Token Decimals | Warning | LOW | ACKNOWLEDGED | p. 80 |
| L–06 | removeLiquidity Griefing Via token0/token1 Caps | DoS | LOW | RESOLVED | p. 81 |
| L–07 | computeRatioX96 Returns Inverted Extreme Prices | Logical Error | LOW | RESOLVED | p. 82 |
| L–08 | Fee Overflow from Unsafe Downcast | Warning | LOW | RESOLVED | p. 83 |

# Findings continued.

| ID | TITLE | CATEGORY | SEVERITY | STATUS | PAGE |
|---|---|---|---|---|---|
| I-01 | Token Configuration Access Control Suggestion | Informational | INFORMATIONAL | RESOLVED | p. 84 |
| I-02 | Misleading Documentation | Documentation | INFORMATIONAL | RESOLVED | p. 85 |
| I-03 | Unexpected ExchangeFee Application | Warning | INFORMATIONAL | ACKNOWLEDGED | p. 86 |
| I-04 | Lacking Division By Zero Validation | Validation | INFORMATIONAL | RESOLVED | p. 87 |
| I-05 | Redundant PoolHook Checks in directSwap | Gas Optimization | INFORMATIONAL | RESOLVED | p. 88 |
| I-06 | Outdated Trusted Forwarder Comments | Documentation | INFORMATIONAL | RESOLVED | p. 89 |
| I-07 | Inaccurate _executePoolFeeHook Documentation | Documentation | INFORMATIONAL | RESOLVED | p. 90 |
| I-08 | Partial Fills For Exact Swaps May Be Misleading | Warning | INFORMATIONAL | RESOLVED | p. 91 |
| I-09 | Misleading _updateFixedPoolHeights Variables | Documentation | INFORMATIONAL | RESOLVED | p. 92 |
| I-10 | Zero Amount Actions | Logical Error | INFORMATIONAL | RESOLVED | p. 93 |
| I-11 | Returned Boolean Parameter Not Used | Superfluous Code | INFORMATIONAL | RESOLVED | p. 94 |
| I-12 | Incorrect Natspec For collectFees | Documentation | INFORMATIONAL | RESOLVED | p. 95 |
| I-13 | Underflow In computeRatioX96 For Large amount1 | Logical Error | INFORMATIONAL | RESOLVED | p. 96 |
| I-14 | Liquidity Removal Hooks Cannot Charge Fees | Informational | INFORMATIONAL | RESOLVED | p. 97 |
| I-15 | Misleading ErrorMsg in _requireCallerIsRegistry | Error | INFORMATIONAL | RESOLVED | p. 98 |
| I-16 | Top only node self linking edge case | Unexpected Behavior | INFORMATIONAL | ACKNOWLEDGED | p. 99 |

# Findings continued.


| ID | TITLE | CATEGORY | SEVERITY | STATUS | PAGE |
|---|---|---|---|---|---|
| I-17 | Equal division allows domination of the pot | Logical Error | INFORMATIONAL | ACKNOWLEDGED | p. 101 |
| I-18 | addInRange False Can Still Produce Fill | Unexpected Behavior | INFORMATIONAL | ACKNOWLEDGED | p. 103 |
| I-19 | Dynamic-fee pool creation improperly blocked | Logical Error | INFORMATIONAL | ACKNOWLEDGED | p. 104 |
| **REMEDIATION REVIEW** | | | | | |
| C-01 | Non Head Order Closing Traps Maker Funds | Logical Error | CRITICAL | RESOLVED | p. 106 |
| C-02 | Pricing Bounds Check Breaks Direct Swaps | DoS | CRITICAL | RESOLVED | p. 108 |
| C-03 | Height List Corruption Breaks Swaps | DoS | CRITICAL | RESOLVED | p. 110 |
| H-01 | Hook Fee Accounting Error | Logical Error | HIGH | RESOLVED | p. 112 |
| H-02 | Side Mix Precision Issue In The Quoter | Logical Error | HIGH | RESOLVED | p. 113 |
| H-03 | Pool Creators Can Avoid Paying Protocol Fees | Validation | HIGH | RESOLVED | p. 115 |
| M-01 | Position Hook Used To Avoid Token Ruleset | Gaming | MEDIUM | RESOLVED | p. 117 |
| M-02 | requireExecutorIsPayer Can Be Bypassed | Validation | MEDIUM | RESOLVED | p. 118 |
| M-03 | Partial Fills Have Insufficient MEV Protection | MEV | MEDIUM | ACKNOWLEDGED | p. 119 |
| L-01 | Partial Fill Fee Overcharge | Logical Error | LOW | ACKNOWLEDGED | p. 120 |
| L-02 | Liquidity positionId Manipulation | Validation | LOW | ACKNOWLEDGED | p. 121 |
| L-03 | Permit Cosignatures Miss Important Fields | Logical Error | LOW | RESOLVED | p. 122 |
| L-04 | Permit Cosignatures Are Not Consumed | Logical Error | LOW | RESOLVED | p. 123 |

# Findings continued.


| ID | TITLE | CATEGORY | SEVERITY | STATUS | PAGE |
|---|---|---|---|---|---|
| L-05 | Cosigner Cannot Be Destroyed Across Systems | Warning | LOW | RESOLVED | p. 124 |
| L-06 | Division By Zero Virtual Reserves | Error | LOW | RESOLVED | p. 126 |
| L-07 | Missing Overflow Check | Validation | LOW | RESOLVED | p. 127 |
| L-08 | Output Swaps Allow Pools To Underpay | Validation | LOW | ACKNOWLEDGED | p. 128 |
| I-01 | Users min/max Protections Can Be Bypassed | Documentation | INFORMATIONAL | ACKNOWLEDGED | p. 129 |
| I-02 | Executor Payment Check For directSwap | DoS | INFORMATIONAL | RESOLVED | p. 131 |

CRITICAL C-01 RESOLVED

# Infinite refund vulnerability

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/515f93daf21572eeb96b5be73b3226890c656a7f/src/handlers/clob/libraries/CLOBHelper.sol#L30 |
| REVIEW | Main Review |

## DESCRIPTION

In handlers/clob/libraries/CLOBHelper.sol `closeOrder()`

when the order being closed is not the bucket's current order ( orderId != currentOrderId ), the code refunds the full unfilled amount to the maker but never marks the order as closed, it never zeroes `ptrOrder.inputAmount` in that branch. Because `inputAmount` remains non-zero, the maker can call `closeOrder` again and again to keep receiving the same refund, inflating their internal balance and ultimately draining the contract's token holdings.

```
// CLOBHelper.closeOrder()
} else {
Order storage ptrCurrentOrder = _orderIdToOrder(currentOrderId);
if (ptrOrder.orderNonce > ptrCurrentOrder.orderNonce) {
// @audit , credits full amount
unfilledInputAmount = ptrOrder.inputAmount;
// unlinks from list
bytes32 previousOrder = ptrOrderBucket.previousOrder[orderId];
bytes32 nextOrder = ptrOrderBucket.nextOrder[orderId];
ptrOrderBucket.nextOrder[previousOrder] = nextOrder;
ptrOrderBucket.previousOrder[nextOrder] = previousOrder;
// @audit , but never sets ptrOrder.inputAmount = 0 (or otherwise marks closed)
} else {
revert CLOBTransferHandler__OrderAlreadyFilled();
}
}
```

Note that at the top of `closeOrder` there is a guard:

```
if (ptrOrder.inputAmount == 0) {
revert CLOBTransferHandler__InvalidInputAmount();
}
```

## C-01 Infinite refund vulnerability CONTINUED

But because the non-current orders branch never zeroes `inputAmount`, that guard never trips on subsequent calls, enabling repeated refunds.

Impact: A maker can infinitely mint credit to their `makerTokenBalance[tokenIn]` by repeatedly closing the same non-current order.

Because the outer handler `CLOBTransferHandler.closeOrder()` subsequently does:

```
makerTokenBalance[tokenIn][msg.sender] += unfilledInputAmount;
```

The attacker can then call `withdrawToken(tokenIn, amount)` to pull the real tokens from the contract. The drain is bounded only by the contract's actual token balance ( the funds deposited by users and orders), So the contract can be fully drained

### RECOMMENDATION

Zero the order's amount in the non-current branch, just like the current-order branch does.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/pull/3

### RESOLUTION

Status: Resolved.

CRITICAL C-02

RESOLVED

# closeOrder Fails to Update inputAmountRemaining

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | CLOBHelper.sol: L53 |
| REVIEW | Main Review |

## DESCRIPTION

In the `closeOrder` function, when closing the head order of a given price bucket the code does:

```
unfilledInputAmount = ptrOrderBucket.inputAmountRemaining;
ptrOrder.inputAmount = 0;
ptrOrderBucket.inputAmountRemaining = 0;
_traverseCLOB(ptrOrderBook, ptrOrderBucket, sqrtPriceX96, currentOrderId);
```

• Zeroes out `inputAmountRemaining`,
• Calls `_traverseCLOB` to advance to the next order (which correctly computes and returns the successor's remaining input),
• But discards all of `_traverseCLOB's` return values and never writes the successor's remaining back into storage.

As a result each time the head is closed, the new head's `inputAmountRemaining` remains 0. Now when the bucket's successor order is cancelled the canceller will receive 0 output from their order as the `unfilledInputAmount = ptrOrderBucket.inputAmountRemaining` is assigned as 0.

Furthermore, this can lead to the orderbook being halted with the `CLOBTransferHandler__InsufficientInputToFill` error when such buckets are reached, though this can be overcome with multiple fills.

## RECOMMENDATION

Modify `closeOrder` to capture and persist the returned `inputAmountRemaining` from _traverseCLOB:

```
(,, uint256 updatedInputAmount,) = _traverseCLOB(...);
ptrOrderBucket.inputAmountRemaining = updatedInputAmount;
```

## C-02 closeOrder Fails to Update inputAmountRemaining CONTINUED

**PROOF OF CONCEPT** https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/pull/1/files#diff-69d0be5fda49bad42591e327016fe53081584ef85b7f2b194f852cc54d530fbe

### RESOLUTION

Status: Resolved.

CRITICAL C-03

RESOLVED

# Reserves can be inflated without actual deposit

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/f14e781f817b099c92493bbd90291e422e836f2a/src/modules/AMMModule.sol#L1144 |
| REVIEW | Main Review |

## DESCRIPTION

In `_distributeAndCollectLiquidityTokens`

```
bool nativeValueUsed = _distributeOrCollectLiquidityToken(provider, token0, netAmount0);
nativeValueUsed = nativeValueUsed || _distributeOrCollectLiquidityToken(provider, token1, netAmount1);
if (msg.value > 0 && !nativeValueUsed) {
revert LBAMM__ValueNotUsed();
}
}
```

The code wants to ensure that if a user sends ETH with their transaction, it gets properly used for wrapped native token operations, otherwise revert with LBAMM__ValueNotUsed().

But a Critical assumption about the || operator occurred

Which is the | | operator will execute both sides and then combine the boolean results

The operator `||` apply the common short-circuiting rules. This means that the | | operator short-circuits - if the left side is true, the right side never executes even if it may have side-effects.

So now whenever the first `_distributeOrCollectLiquidityToken` call returns `true`, the function skips the entire second token in the pool processing.

By checking the three protocol pool types addLiquidity logic to know when they ask for both tokens of the pool versus one-sided.

Requiring both tokens means: The pool type returns ( deposit0 > 0 ) and ( deposit1 > 0 ) to the AMM. That tells the AMM to collect both tokens from the LP for this addLiquidity call.

The DynamicPoolType: requires both tokens when the price range straddles the current price.

## C-03 Reserves can be inflated without actual deposit CONTINUED

https://github.com/GuardianOrg/amm-pool-type-dynamic-team1-1757356782140/blob/ea6b4e8a727c35a7ef1624662655e76ea1354eeb/src/DynamicPoolType.sol#L264-L265

The SingleProviderPoolType: provider explicitly sets both amounts; can be both, depending on the caller.

https://github.com/GuardianOrg/lbamm-pool-type-single-provider-team1-1757356753644/blob/ab2d4e05615aba9a43c0fee6e5dcc218503d3703/src/SingleProviderPoolType.sol#L199-L200

In addLiquidity, reserves are adjusted before calling this function. If `token1`'s transfer/collection is skipped, the contract's recorded reserves increase without actually receiving `token1`,

An attacker can add liquidity to a WETH/Token pool with WETH as token0 by sending only ETH (via msg.value) and avoid transferring the required token1, yet still be credited as if both sides were funded. They can then remove liquidity to extract token1 they never deposited

He gets free liquidity on one side, leading to

Draining every WETH/Token Pool in the protocol.

### RECOMMENDATION

Call both branches unconditionally, then OR the results.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/pull/2

### RESOLUTION

Status: Resolved.

CRITICAL C-04

RESOLVED

# Executor spoofing causes users tokens draining

| | |
|---|---|
| CATEGORY | Access Control |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/f14e781f817b099c92493bbd90291e422e836f2a/src/modules/AMMModule.sol#L3078 |
| REVIEW | Main Review |

## DESCRIPTION

```
function _collectToken(address provider, address token, uint256 amount) internal {
if (amount == 0) return;
uint256 balanceBefore = IERC20(token).balanceOf(address(this));
SafeERC20.safeTransferFrom(token, provider, address(this), amount);
if (IERC20(token).balanceOf(address(this)) != balanceBefore + amount) {
revert LBAMM__TokenInTransferFailed();
}
}
```

`_collectToken` blindly uses the caller-supplied provider as the `from` address for `transferFrom`. And that `provider` is not constrained to `msg.sender`. In the flash-loan path, _collectToken is invoked with `flashloanRequest.executor`

```
// _flashLoan()
ILimitBreakAMMFlashloanCallback(flashloanRequest.executor).flashloanCallback();
// If balances are short, forcibly pull from the 'executor'
if (tokenBalanceAfter < requiredTokenBalanceAfter) {
_collectToken(flashloanRequest.executor, flashloanRequest.loanToken,
requiredTokenBalanceAfter - tokenBalanceAfter);
}
if (tokenBalanceAfter < requiredFeeTokenBalanceAfter) {
_collectToken(flashloanRequest.executor, feeToken,
requiredFeeTokenBalanceAfter - tokenBalanceAfter);
}
```

The `flashloanRequest.executor` is fully caller-controlled and never required to equal `msg.sender`. This creates an allowance-drain/griefing vector.

Attacker sets victim as executor, AMM pulls back the given loanAmount + fee from victim each time, draining their balance via repeated fee siphoning.

## C-04 Executor spoofing causes users tokens draining CONTINUED

Keep repeating to drain all users' token balances which had been granted full allowances to the protocol

### RECOMMENDATION

Bind `executor` to the caller.

```
// In _flashLoan
if (flashloanRequest.executor != msg.sender) {
revert LBAMM__InvalidExecutor();
}
```

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/pull/3

### RESOLUTION

Status: Resolved.

CRITICAL C-05

RESOLVED

# Fee Growth Misaccounting in Fixed Pools

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | `FixedPools.sol, FixedHelper.sol` |
| REVIEW | Main Review |

## DESCRIPTION

In Fixed Pools, fee growth is tracked relative to the `currentHeight` (analogous to Uniswap's tick). Heights are initialized based on the following convention:

- Case 1: `height ≤ currentHeight` → `feeGrowthOutside = feeGrowthGlobal`
- Case 2: `height > currentHeight` → `feeGrowthOutside = 0`

When a swap crosses a height (`_crossHeight`), feeGrowthOutside is flipped:

```
specificHeightInfo.feeGrowthOutside0X128 =
heightCache.feeGrowthGlobalOf0X128 - specificHeightInfo.feeGrowthOutside0X128;
```

This ensures that Case1 / Case 2 are adhered to after a swap, to ensure correct fee attribution.

However, if `currentHeight` lands exactly at a height boundary (i.e., equal to `nextHeightAbove` or `nextHeightBelow`), then positions whose liquidity boundary matches this `currentHeight` will be incorrectly classified:

- In a `zeroForOne = true` swap (height decreasing), Case 1 is treated as Case 2.
- This misclassification causes fee growth to be calculated as if fees were accrued outside the boundary when they should have been inside.

Impact:
- Liquidity providers who collect fees in this precise situation can end up collecting lesser or zero fees.
- Furthermore, an attacker can strategically swap to move `currentHeight` to the exact boundary, and exploit this issue to steal fees from other LPs (see POC attached).

See [source](https://github.com/code-423n4/2021-09-sushitrident-2-findings/issues/1) for a more detailed explanation on Uniswap fee growth mechanism and handling of the boundary case.

## C-05 Fee Growth Misaccounting in Fixed Pools CONTINUED

### RECOMMENDATION

In Uniswap, this is avoided by explicitly adjusting the tick after zeroForOne swaps see [here] (https://github.com/Uniswap/v3-core/blob/main/contracts/UniswapV3Pool.sol#L725).

For Fixed Pools, since `remainingAtHeight` liquidity must be accounted for, a direct adjustment is less trivial. One option is to revert when `currentHeight` lands on a boundary during `zeroForOne = true` swaps:

```
if (heightCache.nextHeightAbove == heightCache.currentHeight) {
revert FixedPool__InvalidPostSwapHeight();
}
```

(at [FixedHelper.sol:1145](https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L1145))

This prevents exploitation, but may cause occasional UX friction if a normal swap coincidentally lands on the boundary.

**PROOF OF CONCEPT** https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/pull/7/files

### RESOLUTION

Status: Resolved.

CRITICAL C-06

RESOLVED

# Filled Orders Remain Cancelable in CLOB

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/main/src/handlers/clob/libraries/CLOBHelper.sol#L203 |
| REVIEW | Main Review |

## DESCRIPTION

After an order is filled in the CLOB, its `inputAmount` is not cleared. The CLOB traversal simply advances to the next order, incrementing the currentOrder's nonce, while leaving the filled order in place.

A `closeOrder` check was intended to prevent cancelling already-filled orders by requiring that the target order's nonce be greater than the currentOrder's nonce:

```
if (ptrOrder.orderNonce > ptrCurrentOrder.orderNonce) {
// perform refund
} else {
revert CLOBTransferHandler__OrderAlreadyFilled();
}
```

However, there is a workaround to this. If the filled order was the final order in the circular linked list, then after advancing, `currentOrder.orderNonce` becomes zero. This bypasses the protection and allows the attacker to cancel a filled order, reclaiming its `inputAmount`.

Attack scenario:

- Attacker creates two orders Order1, Order2 (needs two orders because Order1 has a nonce of 0 which blocks the attack).
- Order2 is large. Attacker executes direct swap that fill his own orders. Orders are filled, but Order2 remains cancellable.
- Attacker cancels Order 2 and receives the refund in his `makerTokenBalance` mapping -- however he cannot withdraw yet due to a lack of liquidity
- Once other users deposit, he can then `withdrawToken` to extract the refunded tokens

## RECOMMENDATION

Clear `order.inputAmount` immediately after an order is filled.

## C-06 Filled Orders Remain Cancelable in CLOB CONTINUED

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/pull/11/files

### RESOLUTION

Status: Resolved.

CRITICAL C-07

RESOLVED

# Head close corrupts global currentPrice

| | |
|---|---|
| CATEGORY | DoS |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/515f93daf21572eeb96b5be73b3226890c656a7f/src/handlers/clob/libraries/CLOBHelper.sol#L258 |
| REVIEW | Main Review |

## DESCRIPTION

Every price sqrtPriceX96 has a bucket that holds a FIFO queue of orders at that exact price

Buckets are linked into an ascending list using two mappings

`nextPriceAbove[price]`, the next higher price ( or a sentinel if there is none )

`nextPriceBelow[price]`, the next lower price ( or a sentinel if there is none )

These sentinels are not real prices, just list end markers, the sentinels here are

0 means "there is no lower price"

`type(uint160).max` means "there is no higher price"

The `currentPrice` global pointer role is to point to the lowest active price level, fill algorithm always starts from there and walks upward

`_traverseCLOB` function is called from `fillOrder` on the current bucket and

`closeOrder` on any bucket where the caller order is the head of that bucket, But not necessarily the current / lowest bucket

_traverseCLOB unconditionally does

```
ptrOrderBook.currentPrice = nextSqrtPriceX96;
```

Because this line is executed whenever the last order in a price bucket is removed, regardless of which bucket that is

a maker can close their head order in any higher priced bucket and forcibly shift the global `currentPrice` to that bucket next above price which will be the MAX sentinel

Because the `nextPriceAbove[thatBucket]` is the MAX sentinel, no price is above it

`_traverseCLOB` then sets

```
ptrOrderBook.currentPrice = type(uint160).max;
```

## C-07 Head close corrupts global currentPrice CONTINUED

`fillOrder` starts with

```
uint160 currentPrice = ptrOrderBook.currentPrice;
if (currentPrice == 0 || currentPrice == type(uint160).max) {
revert CLOBTransferHandler__InvalidPrice();
}
```

From now on, any `ammHandleTransfer` call to `fillOrder` reverts with `InvalidPrice()` as long as `currentPrice` is that sentinel

This can lead to a continuous DoS scenario

No swaps can fill against this order book until someone opens a new order that ( sets currentPrice < MAX )

at which point the attack can be repeated again and again. No cost to the attacker just gas, he can use any minimum order amount which he will get back on close

Breaking the orderbook

### RECOMMENDATION

Only move the global pointer if we just removed the current head level

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/6dd4fffa9e968645b04ee7b37affa4c2a6931659/test/handlers/clob/ClobTransferHandler_CurrentPricePointerDoS_PoC.t.sol

### RESOLUTION

Status: Resolved.

CRITICAL C-08

RESOLVED

# Fixed Pool Height Pointer DOS

| | |
|---|---|
| CATEGORY | DoS |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L492 |
| REVIEW | Main Review |

## DESCRIPTION

`_removeLiquidity` only refreshes `height.nextHeightAbove/nextHeightBelow` when the pool's `currentHeight` lies inside the removed interval.

```
_removeLiquidity()
if (currentHeight >= startHeight && currentHeight < endHeight) {
// updates height state
}
// else does no further updates
```

If a position that sat strictly above (or below) the current height is withdrawn, `_removeLiquidityFromHeight` flips that height out of `heightMap`, but `_removeLiquidity` leaves the global pointers pointing at the now-empty height.

As a result, the next swap (or any call into `_increaseHeight`) that needs to traverse past that stale pointer tries to cross into a height whose liquidityGross is already zero. `_crossHeight` picks up `nextHeightAbove == 0`.

Next, when `_increaseHeight` calculates `(nextHeightAbove - currentHeight) * liquidity`, the subtraction underflows and the transaction panics. From that point on, every upward swap reverts. Liquidity above the broken height is inaccessible until someone re-adds a range that reinitializes the height node. The pool remains DoS'd indefinitely and can be re-broken after each manual repair.

Update: This issue only surfaces after implementing the recommended fix for M-04.

## RECOMMENDATION

In `_removeLiquidity`, update `height.nextHeightAbove` and `height.nextHeightBelow` whenever the corresponding start or end height flips, regardless of where `currentHeight` sits. Reuse the `nextBelow`/`nextAbove` values returned by `_removeLiquidityFromHeight` to rewrite the pointers (mirroring how `_addLiquidity` is implemented).

## C-08 Fixed Pool Height Pointer DOS CONTINUED

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-core-fuzz-1757356641758/pull/15/files#diff-55add19251802f0e16856a973337384ba227d3e51d53e629fc456ed4a6992a95R1208

### RESOLUTION

Status: Resolved.

HIGH **H-01** RESOLVED

# Incorrect Linked List Update in `_traverseCLOB`

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/main/src/handlers/clob/libraries/CLOBHelper.sol#L257 |
| REVIEW | Main Review |

## DESCRIPTION

The _traverseCLOB function assumes that the provided `sqrtPriceX96` is the current price of the orderbook, however this is not always the case.

In the `closeOrder` function `_traverseCLOB` is invoked when the `orderId == currentOrderId`, however the `currentOrderId` is simply the first order in the bucket belonging to the arbitrary `sqrtPriceX96` provided by the user which houses their order.

As a result, when this is the only order in said bucket, the `ptrOrderBook.nextPriceBelow[nextSqrtPriceX96]` entry is assigned to 0. This is okay when the current price bucket being processed is the active price in the orderbook, since the orderbook has a single direction in which it is filled. However when the bucket is not the active one there are indeed buckets below the next price bucket that should be written in the `ptrOrderBook.nextPriceBelow[nextSqrtPriceX96]` entry but are not.

As a result of the `ptrOrderBook.nextPriceBelow[nextSqrtPriceX96]` entry being errantly assigned to zero, the insertion of future orders can perturb the linked order list in such a way that skips one or more bucket of orders.

Consider the following example:

- Bucket 90 is at price 90, Bucket 100 is at price 100, Bucket 110 is at price 110, Bucket 120 is at price 120
- Each bucket has one order in it
- The active price is at 90
- The owner of Bucket 110's order cancels their order, ptrOrderBook.nextPriceBelow[120] is assigned to 0
- User A creates an order at price 80 and provides a hint of price 100
- nextPriceAbove from the hint of 100 is 120
- nextPriceBelow from the bucket 120 is 0
- 80 meets the criteria of 0 < 80 < 120, so the insertion logic gives the price of 80 a nextPriceAbove of 120 and a nextPriceBelow of 0

## H-01 Incorrect Linked List Update in _traverseCLOB CONTINUED

• Now when the price 80 bucket is filled, it skips over all other buckets and goes straight to the nextPriceAbove of the 120 bucket

### RECOMMENDATION

Remove the assumption that the `sqrtPriceX96` value and the provided bucket are the current ones for the order book in the `_traverseCLOB` function with the following assignments:

```
if (nextOrderId == bytes32(0)) {
nextSqrtPriceX96 = ptrOrderBook.nextPriceAbove[sqrtPriceX96];
ptrOrderBook.nextPriceAbove[ptrOrderBook.nextPriceBelow[sqrtPriceX96]] = nextSqrtPriceX96;
/// update with next 3 lines
ptrOrderBook.nextPriceBelow[nextSqrtPriceX96] = ptrOrderBook.nextPriceBelow[sqrtPriceX96];
ptrOrderBook.nextPriceAbove[sqrtPriceX96] = 0;
ptrOrderBook.nextPriceBelow[sqrtPriceX96] = 0;
///
ptrOrderBook.currentPrice = nextSqrtPriceX96;
ptrUpdatedOrderBucket = ptrOrderBook.priceOrderBucket[nextSqrtPriceX96];
ptrUpdatedOrder = _orderIdToOrder(ptrUpdatedOrderBucket.currentOrderId);
inputAmountRemaining = ptrUpdatedOrderBucket.inputAmountRemaining;
}
```

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/pull/2/files

### RESOLUTION

Status: Resolved.

HIGH **H-02** RESOLVED

# Price shifts cause unbounded deposits

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/f14e781f817b099c92493bbd90291e422e836f2a/src/modules/AMMModule.sol#L384 |
| REVIEW | Main Review |

## DESCRIPTION

`_positionAddLiquidity` lets the contract pull unbounded token amounts from the LP given allowance.

After `PoolType.addLiquidity` returns `deposit0` and `deposit1`

the function collects from the provider:

```
function _collectToken(address provider, address token, uint256 amount) internal {
if (amount == 0) return;
uint256 balanceBefore = IERC20(token).balanceOf(address(this));
SafeERC20.safeTransferFrom(token, provider, address(this), amount);
if (IERC20(token).balanceOf(address(this)) != balanceBefore + amount) {
revert LBAMM__TokenInTransferFailed();
}
}
```

When adding liquidity, users must be able to cap how much of each token the contract is allowed to take.

The current code enforces a minimum on `deposit0` / `deposit1`. That lower bound provides no protection against the pool returning very large `deposit` values when price/state movement changes between the user's signing time and execution, and since the user would give full allowance/approval, the call will succeed.

The contract can take arbitrarily larger amounts than the user intended, causing a loss for the user

## RECOMMENDATION

Allow the user to choose a max deposit slippage to consider price movement cases.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/pull/1

## H-02 Price shifts cause unbounded deposits CONTINUED

### RESOLUTION

Status: Resolved.

HIGH **H-04** RESOLVED

# Mapping collision merges distinct balances

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/f14e781f817b099c92493bbd90291e422e836f2a/src/modules/AMMModule.sol#L2768 |
| REVIEW | Main Review |

## DESCRIPTION

`tokensOwed` re-uses the same key shape for two different accounting domains,

That shows up in `_storeTokensOwed` via the `owedTo` argument.

The bug is that direct fees and position debts use identical key shapes:

```
keccak256(tokenFor, tokenFee) // direct fees (token-managed)
keccak256(owedTo, tokenOwed) // position debts
```

So whenever `owedTo == tokenFor` and `tokenOwed == tokenFee`, both balances are written to/read from the exact same slot. `_storeTokensOwed` therefore writes into the direct-fee bucket whenever the owed recipient happens to be that token's address.

If a token contract address Y ever becomes the LP provider ( common when a project seeds liquidity ) and a distribution transfer fails, `_storeTokensOwed(Y, USDC, x)` adds to the same entry that holds contract X token-managed hook fees in `USDC`.

From then on, we cannot distinguish "fee pot" vs "owed debt" in that slot.

## RECOMMENDATION

Use distinct prefixes or separate mappings, so the key spaces cannot collide.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/pull/6

## RESOLUTION

Status: Resolved.

HIGH **H-05**

PARTIALLY RESOLVED

# Partial Fills Invalidate Limit Validation

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | `AMMModule.sol` |
| REVIEW | Main Review |

## DESCRIPTION

During the `exactIn` and `exactOut` pool swap flows, partial fills are supported which allow the input amount and output amount to be only a portion of what was requested respectively.

However when a partial fill takes place there is no adjustment made to the `swapOrder.limitAmount`, therefore a trader may unintentionally accept a much worse price execution when a partial fill takes place.

## RECOMMENDATION

Consider adjusting the `limitAmount` by the proportion that the order was partially fulfilled to maintain the same execution price limit for the trader.

## RESOLUTION

Status: Partially Resolved.

HIGH **H-06** RESOLVED

# Wrong Function Used in _updateFixedPoolHeights

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L1022 |
| REVIEW | Main Review |

## DESCRIPTION

In `FixedPools._updateFixedPoolHeights`, liquidity heights and reserves are updated after a fixed price swap. For `zeroForOne` swaps, the following calculation is performed:

```
uint256 position0ShareOf1 = calculateFixedOutput(ptrPoolState.height0.consumedLiquidity, sqrtPriceX96, true);
```

This is intended to represent how much of token0's `consumedLiquidity` can be used to fulfill the swap. However, the function incorrectly calls `calculateFixedOutput` instead of `calculateFixedInput`. Since `consumedLiquidity` is being passed as the exact input, the correct method should be `calculateFixedInput`.

A similar issue exists for `oneForZero` swaps when calculating `position1ShareOf0`.

As a result, `position0ShareOf1` (and symmetrically, `position1ShareOf0`) is severely undervalued. This skews the fee apportionment logic. `Token0 (or token1)` liquidity providers receive less fees than they are entitled to, leading to systematic loss for one side of LPs.

## RECOMMENDATION

Replace `calculateFixedOutput` with `calculateFixedInput` in:

- `zeroForOne` case: calculating `position0ShareOf1`
- `oneForZero` case: calculating `position1ShareOf0`

**PROOF OF CONCEPT** https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/pull/2/files

## RESOLUTION

Status: Resolved.

HIGH **H-07** RESOLVED

# Position Side Not Cleared For Empty Ranges

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L158-L181 |
| REVIEW | Main Review |

## DESCRIPTION

The `depositLiquidity` function calls `_collectPosition` to collect and return all accrued fees. However, if the side's computed range is empty ( `startHeight == endHeight` ), the position's side metadata and fee growth checkpoints are not cleared or updated as the `_addLiquidity` function is never executed.

Consequently, the following impacts were detected:

• When adding liquidity, fees are actually claimed and used to offset the required tokens to deposit in the `_distributeAndCollectLiquidityTokens` call. This enables a subsequent `collectFees` call to claim the same fees again for that side.
• Sudden increase in position value, as it still counts liquidity from the empty range side.
• `liquidityGross` can underflow in `_removeLiquidityFromHeight` as it tries to remove liquidity from a height that no longer exists (empty range).
• `feeBalanceX` can revert with `SafeCast__Uint128Overflow` as it tries to remove fees again from `poolState.feeBalanceX`.

## RECOMMENDATION

Mirror the cleanup performed in withdrawLiquidity: when `liquidityCache.startHeightX == liquidityCache.endHeightX`, explicitly clear the position's side X metadata and last fee growth checkpoints.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/pull/1/files#diff-e4ae8637ec25bef676b122b3cad2de5c0be5ba5fc351c98c29c619cfb22bfb24

## RESOLUTION

Status: Resolved.

HIGH **H-08** RESOLVED

# addLiquidity Frontrun via Height Manipulation

| | |
|---|---|
| CATEGORY | Frontrunning |
| LOCATION | `FixedPoolType.sol:199` |
| REVIEW | Main Review |

## DESCRIPTION

In Fixed Pools, liquidity is always added around the current height. This creates a frontrun vector:

1. Alice (attacker) sees Bob preparing to add liquidity.
2. Alice frontruns by swapping to push `currentHeight` to an extreme height.
3. Bob's transaction executes, and his liquidity is placed at this manipulated (very high) height.
4. Alice backruns with another swap to return `currentHeight` to the normal range.

As a result, Bob's liquidity sits at a distant, inactive height and earns no fees under normal conditions. Whales with large positions can deliberately manipulate heights to force smaller users' liquidity into useless ranges, effectively monopolizing fee collection.

## RECOMMENDATION

When adding liquidity, allow users to specify a maximum acceptable height. If `currentHeight` exceeds this bound during execution, the transaction reverts, protecting against manipulation.

Additionally, users should be encouraged to submit liquidity transactions via private mempools where possible, to reduce exposure to frontrunning and manipulation.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/pull/11/files

## RESOLUTION

Status: Resolved.

HIGH **H-09** RESOLVED

# Token hook fees can be totally bypassed

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/1b566b0379738a708c225fa2ea779378937f6326/src/hooks/AMMStandardHook.sol#L95 |
| REVIEW | Main Review |

## DESCRIPTION

In LimitBreakAMM.sol, the `multiSwap` function calls `_initializeSwapCache`. This function populates `swapCache.context` with the tokenIn and tokenOut from the top-level "swapOrder"

This context is never updated for subsequent hops.

The `multiSwap` function loops through pools and calls `_poolExactInputSwap` or `_poolExactOutputSwap`.

These functions then call `_executeBeforeSwapHooks` and `_executeAfterSwapHooks`, which in turn call `_executeSwapHook`.

While `_executeSwapHook` correctly packs the current hop's tokens into the "HookSwapParams" part of the calldata, the "SwapContext" part remains unchanged and reflects the overall swap's start and end tokens.

This is exploitable in the the AMMStandardHook.sol because

Its "beforeSwap" and "afterSwap" functions determine which token settings to apply based on `context.tokenIn` and `context.tokenOut`, not the `swapParams`

```
// in AMMStandardHook.sol
function beforeSwap() external returns (uint256 fee) {
// @audit, 'context.tokenIn' and 'context.tokenOut' are from the top-level swap,
// not the current hop's tokens which are in 'swapParams'.
(address token, address pairToken) =
swapParams.hookForInputToken ? (context.tokenIn, context.tokenOut) : (context.tokenOut, context.tokenIn);
// It fetches settings based on the wrong tokens
HookTokenSettings memory tokenSettings = _getOrFetchTokenSettings(token);
// and calculates a fee which is then applied to the current hop's amount.
if (swapParams.exactInputSwap) {
if (swapParams.hookForInputToken) {
```

## H-09 Token hook fees can be totally bypassed CONTINUED

```
fee = _calculateFee(swapParams.amount, tokenSettings.tokenFeeSellBPS);
}
}
}
```

An attacker can exploit this to avoid paying fees on a specific token.

ZERI is a token with a 10% swap fee configured via AMMStandardHook, while WETH and USDC have no fees.

Three pools exist: `WETH/BNC`, `BNC/ZERI`, and `ZERI/USDC`

The attacker executes a `multiSwap` from WETH → BNC → ZERI → USDC,

Hop 1 ( WETH → BNC ): Hooks use WETH/USDC context, no fees applied.

Hop 2 ( BNC → ZERI ): Hooks still see WETH/USDC context, ZERI's 10% fee is bypassed.

Hop 3 ( ZERI → USDC ): Same stale context, ZERI fee bypassed again.

The attacker successfully swapped through `ZERI` twice without paying any of its intended 10% fee. The protocol and `ZERI` holders lose all expected revenue from this trade

### RECOMMENDATION

Not recorded.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/1b566b0379738a708c225fa2ea779378937f6326/test/StaleContextFeeEvasion.t.sol

### RESOLUTION

Status: Resolved.

HIGH **H-10** RESOLVED

# 16-bit overflow in the protocol fee denominator

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/f14e781f817b099c92493bbd90291e422e836f2a/src/modules/AMMModule.sol#L2506 |
| REVIEW | Main Review |

## DESCRIPTION

In both `_applyExactInputInputFees` and `_applyExactOutputInputFees` the code compute the grossed-up amount that must be taken from `amountIn` to satisfy the minimum protocol fee via

```
protocolFeeFromInput = FullMath.mulDivRoundingUp(
shortage,
DOUBLE_BPS,
(DOUBLE_BPS - poolFeeBPS * swapCache.protocolFeeStructure.lpFeeBPS)
);
```

`poolFeeBPS` is a `uint16`

`swapCache.protocolFeeStructure.lpFeeBPS` is also `uint16`.

Because both operands are `uint16`, the product `poolFeeBPS * lpFeeBPS` is computed in 16-bit arithmetic. Inside the `unchecked` block this silently wraps modulo 2^16

The result is then upcast to `uint256` for the subtraction with `DOUBLE_BPS`. That makes the denominator wrong by orders of magnitude

Minimum protocol fee can be under-collected

## RECOMMENDATION

We could replace the denominator computation with a properly widened multiply, by casting to `uint256` before multiplying

```
uint256 poolBps = uint256(poolFeeBPS);
uint256 lpBps = uint256(swapCache.protocolFeeStructure.lpFeeBPS);
uint256 product = poolBps * lpBps; // fits safely, 10_000 * 10_000 = 1e8
uint256 denom = DOUBLE_BPS - product;
protocolFeeFromInput = FullMath.mulDivRoundingUp(shortage, DOUBLE_BPS, denom);
```

## H-10 16-bit overflow in the protocol fee denominator CONTINUED

### RESOLUTION

Status: Resolved.

HIGH **H-11** RESOLVED

# Predictable IDs enable whitelist hijacking

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | `creatorHookSettingsRegistry.sol` |
| REVIEW | Main Review |

## DESCRIPTION

`setTokenSettings` does not verify whitelist ID existence/ownership, enabling attackers to later create / own that ID and hijack the token's whitelists via the registry and hook sync path

Token settings can point to any not yet created whitelist ID

`setTokenSettings()` blindly stores the `HookTokenSettings` struct, including `pairedTokenWhitelistID` and `lpWhitelistID` without verifying that those list IDs exist, and are controlled by the caller or are immutable/renounced

Because whitelist IDs are globally auto-incremented and anyone can call `createPairTokenWhitelist` / `createLpWhitelist`, an attacker can late-bind themselves to a token's settings, example

A token admin calls `setTokenSettings` with `settings.pairedTokenWhitelistID = x` and / or `lpWhitelistID = x`

Where x doesn't exist yet

An attacker watches chain mempool, then spams `createPairTokenWhitelist` or the LP version until `_nextPairTokenListId` reaches x, thereby becoming the owner of ID x

The attacker populates the list with his arbitrary addresses via `updatePairTokenWhitelist()` and then calls it with `hooksToSync` that include the AMM hooks used by the victim token

Attacker now controls which pair tokens/LPs/pool types the hook will treat as whitelisted for that token

Root cause, `setTokenSettings()` never checks ownership or existence of the referenced list IDs, it just stores them and optionally notifies hooks

Whitelist creation is permissionless and sequential, IDs are predictable and can be raced to by an attacker

## H–11 Predictable IDs enable whitelist hijacking CONTINUED

Hooks trust the registry, so whoever owns the list ID can push content to any hook via `hooksToSync`

### RECOMMENDATION

Add existence or ownership checks in `setTokenSettings()` before writing settings or syncing hooks

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/94a37d7251b5c8a88c3b3c0b882d166a19f51272/test/LateBindingWhitelistHijack.t.sol

### RESOLUTION

Status: Resolved.

HIGH **H-12** RESOLVED

# Uninitialized tokens allow whitelist bypass

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/515f93daf21572eeb96b5be73b3226890c656a7f/src/hooks/AMMStandardHook.sol#L210 |
| REVIEW | Main Review |

## DESCRIPTION

If a token's hook settings are not initialized in the registry, the hook silently self initializes to fully permissive defaults ( no whitelists, trading not paused )

Pool pair whitelist is only enforced at pool creation time, not at swap time because the swap hooks do not check pairedTokenWhitelistID, they only check paused/price bounds

This allow this scenario

Token owner enables the pool-creation hook flag, but hasn't yet called the registry to sync token settings

A user immediately creates any pool for that token with any pair/pool-type/fee

Because registry says uninitialized , `getOrFetchTokenSettings` returns permissive defaults, so validatePoolCreation allows it.

Later, the token owner finally syncs token settings with pair whitelists.

But it's too late, pair whitelist is only checked at creation, not during swaps.

The already created, unauthorized pool can continue to trade forever because beforeSwap/afterSwap don't enforce the pair whitelist

This leads to Irreversible bypass of pair restrictions for any pools created in the initialization window

## RECOMMENDATION

Hooks should reject by default for uninitialized tokens, for example default to tradingIsPaused = true or revert pool creation/swaps until settings are synced

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/1917ed3db5ad6c2f429c50403ca927f77262d6b9/test/AMMStandardHookPoCs.t.sol

## H–12 Uninitialized tokens allow whitelist bypass CONTINUED

### RESOLUTION

Status: Resolved.

HIGH **H-13** RESOLVED

# Tick crossing without reaching boundary

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/amm-pool-type-dynamic-team1-1757356782140/blob/ea6b4e8a727c35a7ef1624662655e76ea1354eeb/src/libraries/DynamicHelper.sol#L288 |
| REVIEW | Main Review |

## DESCRIPTION

The code can wrongly cross a tick even when the swap only moved to the user price limit and did not reach the next tick price

In both swap entry points exactInputSwap / exactOutputSwap

The code accept a user supplied limit price when `swapExtraData.length == 32`

This is fine by itself, but the downstream swap

mixes up "price at the next tick boundary" with

"price target capped by the limit"

In `DynamicHelper.computeSwap`

```
// assumes reaching step.sqrtPriceNextX96 means we reached the next tick
// even if step.sqrtPriceNextX96 was the limit inside the current tick
if (swapCache.sqrtPriceCurrentX96 == step.sqrtPriceNextX96) {
if (step.initialized) {
int128 liquidityNet = _crossTick();
if (swapCache.zeroForOne) liquidityNet = -liquidityNet;
swapCache.liquidity = LiquidityMath.addDelta(swapCache.liquidity, liquidityNet);
}
swapCache.tick = swapCache.zeroForOne ? step.tickNext - 1 : step.tickNext;
}
```

`step.sqrtPriceNextX96` is misused for two different meanings

Because `step.sqrtPriceNextX96` was overwritten with the limit, the condition `swapCache.sqrtPriceCurrentX96 == step.sqrtPriceNextX96` becomes true even when the swap stopped at the limit before reaching the next tick. The code then sets `swapCache.tick = step.tickNext`, even though the price never reached that tick

The code falsely triggers a tick crossing, even though the price never reached the tick boundary

## H–13 Tick crossing without reaching boundary CONTINUED

At the end of the swap the code persist this inconsistent state

`(ptrPoolState.sqrtPriceX96, ptrPoolState.tick) = (swapCache.sqrtPriceCurrentX96, swapCache.tick);`

Now the pool `(sqrtPriceX96, tick)` pair is mathematically inconsistent

This state corruption can be chained across steps/swaps to extract value from LPs ( by forcing favorable liquidity before calculating deltas, manipulating fee growth inside/outside ranges )

Once `swapCache.liquidity` has been incorrectly adjusted due to the tick crossing at the limit, this code corrupts the pool active liquidity in storage `pools[poolId].liquidity`

### RECOMMENDATION

We need to not reuse the same variable for "next tick price" and "price target capped by limit" , Keeping them separate and only advance if we reached the tick price

PROOF OF CONCEPT https://github.com/GuardianOrg/amm-pool-type-dynamic-team1-1757356782140/blob/881f6174e91dfac828be178b8002a3506a5ed0a2/test/DynamicPoolLimitBoundaryBug.t.sol

### RESOLUTION

Status: Resolved.

MEDIUM **M-01** RESOLVED

# The pair-token whitelist is enforced wrongly

CATEGORY Logical Error

LOCATION https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/515f93daf21572eeb96b5be73b3226890c656a7f/src/hooks/AMMStandardHook.sol#L218

REVIEW Main Review

## DESCRIPTION

When someone tries to create a new pool (ex. TOKEN <-> USDC) the AMM calls `validatePoolCreation()`

```
function validatePoolCreation(
address creator,
bool hookForToken0,
PoolCreationDetails memory details,
bytes calldata /*hookData*/
) external {
address token = hookForToken0 ? details.token0 : details.token1;
// @audit, 'token' (the hook's own token) is passed as the 'pairToken'
_enforcePoolCreationSettings(details, token, creator, _getOrFetchTokenSettings(token));
}
```

If `hookForToken0` is `true` it picks `details.token0` (Token0 address).

Then the code passes this address to `_enforcePoolCreationSettings`, which expects its second argument to be the other token in the pair.

```
/// @param pairToken The address of the other token in the pair.
function _enforcePoolCreationSettings(
PoolCreationDetails memory details,
address pairToken,
address creator,
HookTokenSettings memory tokenSettings
) internal view {
if (tokenSettings.pairedTokenWhitelistID > 0) {
if (!_pairTokenWhitelists[tokenSettings.pairedTokenWhitelistID].contains(pairToken)) {
revert AMMStandardHook__PairNotAllowed();
}
}
}
```

## M-01 The pair-token whitelist is enforced wrongly CONTINUED

`validatePoolCreation` passes the hook token address itself instead of the paired token.

Impact: All pool creations that require a pair whitelist will always revert with `AMMStandardHook__PairNotAllowed()`, even when the real pair token is whitelisted. That bricks pool creation for tokens configured with a pair whitelist.

The code now is comparing the hook token address against its own token address in the whitelist (which doesn't exist there),

Because the whitelist contains only a list of acceptable pair tokens.

### RECOMMENDATION

Pass the actual counterparty token address into `_enforcePoolCreationSettings`

PROOF OF CONCEPT https://gist.github.com/minato7namikazi/752e8158069975b396c3cbb879a80b30

### RESOLUTION

Status: Resolved.

MEDIUM M-02

RESOLVED

# Users Pay Full Fee On ExactInput Partial Fills

| | |
|---|---|
| CATEGORY | Unexpected Behavior |
| LOCATION | `AMMModule.sol` |
| REVIEW | Main Review |

## DESCRIPTION

During the execution of exactInput the `exchangeFee` is taken at the beginning based on the initial `amountIn` during the `_initializeSwapCache` execution.

However this initial `amountIn` may not be representative of the amount that is actually used in the swap, which may be significantly smaller in the event of a partial fill.

In the event of a partial fill, the `exchangeFee` charged could be exorbitant relative to the amount that was actually filled, and would represent a much larger share of the amount into the swap than the original `exchangeFee.BPS`.

This behavior is in contrast to the exchangeFee logic for exactOut swaps, where the fee is levied on the final result which accounts for partial fills.

## RECOMMENDATION

In the event of a partial fill, consider if the exchangeFee should be re-calculated to reflect the decrease in input tokens for the swap.

## RESOLUTION

Status: Resolved.

MEDIUM M-03

PARTIALLY RESOLVED

# Minimum Protocol Fee May Not Always Be Met

| | |
|---|---|
| CATEGORY | Unexpected Behavior |
| LOCATION | `AMMModule.sol` |
| REVIEW | Main Review |

## DESCRIPTION

During the execution of an `exactIn` pool swap, the `minimumProtocolFee` is defined as the `inputTokenHopFeeBPS` percentage of the `swapAmountIn`. During the application of the `exactInput` input token fees, if the `minimumProtocolFee` is not met the `protocolFeeFromHookFees` is increased to meet it.

However since the `poolFeeBPS` is not guaranteed to be charged by the pool type, this minimum amount is not guaranteed to be met.

The call to `exactInputSwap` on the `poolType` allows the `poolType` contract to define the `poolProtocolFees` and `poolFeeOfAmountIn` such that the fee amounts are not large enough to meet the previously defined `minimumProtocolFee`.

This results in the protocol not charging the minimum amount of fees when pool types do not apply sufficient fees themselves. This is in contrast to `exactOut` swaps where the `minimumProtocolFee` will always be met since the fees charged by the `poolType` are already known.

## RECOMMENDATION

If the `minimumProtocolFee` should always be upheld, then consider validating that the resulting `poolProtocolFees` allow it to be met. This validation could be made for exactIn actions in the `_validateProtocolFees` function.

## RESOLUTION

Status: Partially Resolved.

MEDIUM **M-04** ACKNOWLEDGED

# addLiquidity can get stuck in an infinite loop

| | |
|---|---|
| CATEGORY | DoS |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/d41f3a822992a6b5c5cb34291646cbb71dcec2f6/src/libraries/FixedHelper.sol#L538-L544 |
| REVIEW | Main Review |

## DESCRIPTION

when `informationHeight == toHeight` and the entry for that height has stale links in `heightMap`

`_addLiquidityToHeight` can get stuck in an infinite loop

`_addLiquidityToHeight` begins by incrementing `liquidityGross` and sets `flipped = (liquidityGrossAfter == 1)`

If `flipped == true`, the code expects the height to be newly activated and therefore unlinked in the intrusive list heightMap[] should have both pointers 0

But, in `_removeLiquidityFromHeight`, when a height flips to zero liquidity,

we do not clear that height's own pointers in the common case. the code only clear them in the narrow `else` branch:

```
if (start || fromHeight != nextHeightAbove) {
// @audit, neighbors rewired But mapHeight's own pointers are left intact (stale)
mapBelow.nextHeightAbove = nextHeightAbove;
mapAbove.nextHeightBelow = nextHeightBelow;
} else {
// @audit, only here we clear mapHeight's pointers
mapBelow.nextHeightAbove = nextHeightAbove = nextHeightBelow;
mapHeight.nextHeightBelow = 0;
mapHeight.nextHeightAbove = 0;
}
```

The result is that it's very easy for a height whose `liquidityGross` is now `0` to still have non-zero `nextHeightBelow/nextHeightAbove` in `heightMap`.

Later, when we try to re add liquidity at that same height and pass a hint equal to that height, `_addLiquidityToHeight` sees

## M-04 addLiquidity can get stuck in an infinite loop CONTINUED

`toHeight == informationHeight`,

`heightMap[informationHeight]` has non-zero neighbors (stale), so the empty sentinel branch does not trigger,

The loop body becomes:

```
while (true) {
// no branch matches when toHeight == informationHeight and neighbors are non-zero
// informationHeight is never changed, no break is hit -> infinite loop -> out-of-gas
}
```

That's a DoS on add-liquidity calls ( and can also block withdraw flows that redeposit ), triggered by a perfectly valid hint.

### RECOMMENDATION

Always clear height's own pointers when it flips to zero

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/d43a94ed555f44a68f1a48cb7bfd2574956d361f/test/FixedPoolAddLiquidityDoS.t.sol

### RESOLUTION

Status: Acknowledged.

MEDIUM M-05

RESOLVED

# updateFixedPoolHeight Infinite Loop Due Rounding

CATEGORY Rounding

LOCATION https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L1048

REVIEW Main Review

## DESCRIPTION

In Fixed Pools, during a swap, `_updateFixedPoolHeights` splits the fill between sideZero and sideOne liquidity. For example, when `zeroForOne = false`:

```
function _updateFixedPoolHeights() {
...
uint256 position1ShareOf0 = calculateFixedInput(ptrPoolState.height1.consumedLiquidity, sqrtPriceX96, false); // modified with calculateFixedOutput due to separate bug
uint256 virtualReserve0 = ptrPoolState.position0ShareOf0 + position1ShareOf0;
uint256 amount1FilledByHeight0 = FullMath.mulDiv(amount0, position1ShareOf0, virtualReserve0); // should be renamed amount0FilledByHeight1
uint256 amount0FilledByHeight0 = amount0 - amount1FilledByHeight0;
...
}
```

Suppose both sides have 100 USDC liquidity and `amountOut = 200`.

```
- calculateFixedInput rounds down: position1ShareOf0 = 99.99
- virtualReserve0 = 100 + 99.99 = 199.99
- amount1FilledByHeight0 = 99.99
- amount0FilledByHeight0 = 200 - 99.99 = 100.01
```

This requires extracting 100.01 USDC from sideZero, which only has 100. The mismatch causes an infinite loop in `_increaseHeight` that consumes all gas and reverting the swap.

## RECOMMENDATION

This issue is related to M-06 where the recommendation to base calculations off `expectedReserves` would resolve the issue here too. I.e. Since `expectedReserves` is rounded down, the above scenario becomes:

## M-05 updateFixedPoolHeight Infinite Loop Due Rounding CONTINUED

- calculateFixedInput rounds down: position1ShareOf0 = 99.99
- virtualReserve0 = 100 + 99.99 = 199.99
- amount1FilledByHeight0 = 99.99
- amount0FilledByHeight0 = 200 199.99 - 99.99 = 100.00 // amount0 (expectedReserves rounded down)

**PROOF OF CONCEPT** https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/pull/9/files#diff-54620cc5f6d96cd9a763132ce1acb4533b262a6258059266840a0e0b8c89978b

### RESOLUTION

Status: Resolved.

MEDIUM M-06

RESOLVED

# Rounding Drift Causes Infinite Loop During Swaps

| | |
|---|---|
| CATEGORY | Rounding |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L1026 |
| REVIEW | Main Review |

## DESCRIPTION

In Fixed Pools, token balances are tracked through these variables:

`position0ShareOf0 + position1ShareOf0; (let's call it expectedReserves0)`
`position1ShareOf1 + position0ShareOf1; (let's call it expectedReserves1)`

These values are frequently rounded during `_updateFixedPoolHeights` calculations. Over time, the rounding effect accumulates, causing `expectedReserves0,1` to drift lower than the true reserves.

This becomes an issue during an `exactInputSwap` where `amountOut` exceeds `reserveOut`:

```
if (amountOut > swapCache.reserveOut) {
//attempt an exact output swap for remaining reserves
swapCache.amountOut = swapCache.reserveOut;
...
}
```

In this case, the swap is converted into an `exactOutputSwap` with `amountOut` set to reserves. Because of the drift, `amountOut` can exceed the expected reserves (`expectedReserves0,1`). When `_updateFixedPoolHeights` attempts to allocate liquidity in this situation, the loop inside `_increaseHeight` cannot be satisfied and continues indefinitely, consuming all gas and reverting.

## RECOMMENDATION

In `swapExactInput`, instead of comparing `amountOut` against `reserveOut`, compare it against the expected reserves which can be derived from `FixedPoolState`:

```
function swapExactInput() {
...
+ uint256 expectedReserve;
+ if(zeroForOne){
```

## M-06 Rounding Drift Causes Infinite Loop During Swaps CONTINUED

```
+ uint256 position0ShareOf1 = calculateFixedInput(ptrPoolState.height0.consumedLiquidity, ptrPoolState.sqrtPriceX96, true);
+ expectedReserve = ptrPoolState.position1ShareOf1 + position0ShareOf1;
+ }
+ else{
+ uint256 position1ShareOf0 = calculateFixedInput(ptrPoolState.height1.consumedLiquidity, ptrPoolState.sqrtPriceX96, false);
+ expectedReserve = ptrPoolState.position0ShareOf0 + position1ShareOf0;
+ }
- // if (amountOut > swapCache.reserveOut) { // @audit avoid infinite loops by comparing amountOut agains expectedReservesX
+ if (amountOut > expectedReserve) {
- //attempt an exact output swap for remaining reserves
- // swapCache.amountOut = swapCache.reserveOut;
+ swapCache.amountOut = expectedReserve; // @audit M-06 fix
+ uint256 initialAmountIn = swapCache.amountIn;
...
}
```

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/pull/10/files#diff-988d86a926b7cf4abd5e1834f9e1aa862f8f904404fdf660eea053c6de829ddf

### RESOLUTION

Status: Resolved.

MEDIUM **M-07** RESOLVED

# Double floor enables zero input

| | |
|---|---|
| CATEGORY | Rounding |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-single-provider-team1-1757356753644/blob/ab2d4e05615aba9a43c0fee6e5dcc218503d3703/src/libraries/SingleProviderHelper.sol#L197-L202 |
| REVIEW | Main Review |

## DESCRIPTION

In `SingleProviderHelper.calculateFixedOutput`, the required input is computed with two consecutive `FullMath.mulDiv` calls that both round down.

Because the conversion uses the square-root price ( `sqrtPriceX96` ) twice,

This "double floor" can drive the intermediate to zero for small `amountOut` values whenever `sqrtPriceX96 ≠ Q96` ( price ≠ 1) That lets a taker obtain a non-zero `amountOut` while paying zero input.

```
function calculateFixedOutput(
uint256 amountOut,
uint160 sqrtPriceX96,
bool zeroForOne
) internal pure returns (uint256 amountIn) {
if (zeroForOne) {
// amountIn := amountOut * (Q96 / sqrtPriceX96)^2
amountIn = FullMath.mulDiv(amountOut, Q96, sqrtPriceX96); // floors
amountIn = FullMath.mulDiv(amountIn, Q96, sqrtPriceX96); // floors again
} else {
// amountIn := amountOut * (sqrtPriceX96 / Q96)^2
amountIn = FullMath.mulDiv(amountOut, sqrtPriceX96, Q96); // floors
amountIn = FullMath.mulDiv(amountIn, sqrtPriceX96, Q96); // floors again
}
}
```

For exact output flows, the input must be rounded up so the pool always receives enough.

Example ( token0 - token1, `zeroForOne = true` )

Let `amountOut = 1` , `sqrtPriceX96 = Q96 + 1`

First step, `floor(1 * Q96 / (Q96+1)) = 0`

second step remains `0`.

## M-07 Double floor enables zero input CONTINUED

Result, `amountIn = 0`, yet `amountOut = 1` is delivered.

Fees are computed from `reserveAmountIn = 0`, so no fee either.

A taker can repeatedly call `exactOutputSwap` with tiny `amountOut` and drain `reserveOut` while paying 0 input

### RECOMMENDATION

Round up when converting desired output to required input. Use `FullMath.mulDivRoundingUp` to effectively compute the ceiling of the squared ratio.

### RESOLUTION

Status: Resolved.

MEDIUM M-09

ACKNOWLEDGED

# Asymmetry In FixedPool Encourages JIT Liquidity

| | |
|---|---|
| CATEGORY | Gaming |
| LOCATION | FixedPoolType.sol |
| REVIEW | Main Review |

## DESCRIPTION

In Fixed Pools, `currentHeight` represents the level of liquidity consumed (analogous to Uniswap's `currentTick`, which represents price).

However, unlike Uniswap, performing two symmetrical swaps (e.g., swap 100 in, then swap 100 out) does not revert `currentHeight` back to the same value.

This is because liquidity is split across `sideZero` and `sideOne`. For example:

- A `token0 → token1` swap increases currentHeight1.
- A subsequent `token1 → token0` swap consumes token0 from both `sideZero` and `sideOne`, increasing `currentHeight0` while only slightly reducing `currentHeight1`.

In the POC, (1:1 price, 100 token0 + 100 token1 liquidity):

- Swap1: 10 token0 → token1: `currentHeight0 = 0.00`, `currentHeight1 = 9.99`
- Swap2: 10 token1 → token0: `currentHeight0 = 9.09`, `currentHeight1 = 9.09`
- Swap3: 10 token0 → token1: `currentHeight0 = 8.18`, `currentHeight1 = 18.18`
- Swap4: 10 token1 → token0: `currentHeight0 = 16.52`, `currentHeight1 = 16.52`

As a result, LPs at lower heights stop earning fees once `currentHeight` surpasses their range, incentivizing constant liquidity repositioning. In Uniswap, Just-in-Time (JIT) liquidity requires mempool monitoring and precise timing, with risks of slippage and impermanent loss if miscalculated. In Fixed Pools, none of these risks apply — price is fixed, there is no worse asset to end up holding, and no slippage to account for. An LP can simply back-run swaps, re-add liquidity at the new height, allowing them to consistently capture more fees compared to long-term LPs.

## RECOMMENDATION

Consider additional JIT mitigations, such as:

## M-09 Asymmetry In FixedPool Encourages JIT Liquidity CONTINUED

- Minimum liquidity duration requirements before earning fees.
- Penalties for frequent add/remove liquidity actions.
- Limiting how close new liquidity can be placed to `currentHeight`.

At minimum, clearly document this behavior so LPs are aware of the risks.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/pull/11/files#diff-5d106040112cb66df9e8704564c7dad57c90b3307fbb08e96aeab8adc7e651c6

### RESOLUTION

Status: Acknowledged.

MEDIUM **M-10** RESOLVED

# Pool creation bypasses initial price validation

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/515f93daf21572eeb96b5be73b3226890c656a7f/src/hooks/AMMStandardHook.sol#L210 |
| REVIEW | Main Review |

## DESCRIPTION

`enforcePoolCreationSettings` claims to enforce min/max price bounds but never checks the initial sqrt price from the pool's poolParams

There is no initial-price bounds enforcement at pool creation

only swap-time price bounds are enforced in `validatePricingBounds`

If a pair is allowed and creator passes pool-creation checks, the pool can be initialized at an extreme out-of-bounds price ( encoded in PoolCreationDetails.poolParams for the chosen pool type )

Depending on bounds, subsequent trades in one direction revert while the other direction remains possible, enabling griefing or forcing unfavorable initial liquidity positioning

## RECOMMENDATION

In `validatePoolCreation`, decode the initial sqrt price from details.poolParams and check against registry bounds for both tokens

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/1917ed3db5ad6c2f429c50403ca927f77262d6b9/test/AMMStandardHookPoCs.t.sol

## RESOLUTION

Status: Resolved.

MEDIUM M-11

RESOLVED

# Pause bypass for afterSwap only setup

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/515f93daf21572eeb96b5be73b3226890c656a7f/src/hooks/AMMStandardHook.sol#L134 |
| REVIEW | Main Review |

## DESCRIPTION

The hook enforces tradingIsPaused only in beforeSwap, but not in afterSwap.

That is fine if all tokens will always use both beforeSwap and afterSwap together

but the system allow any token owner to activate only 1 flag per their preference

by calling `setTokenSettings` with packedSettings `2` only which corresponds to TOKEN_SETTINGS_AFTER_SWAP_HOOK_FLAG

If a token is configured to use only the afterSwap hook, and registry sets tradingIsPaused = true for the token and syncs to the hook

A token can end up paused in the registry and still trade

## RECOMMENDATION

Add the same pause enforcement to afterSwap

Or enforce that if a token uses the pause feature, beforeSwap must be enabled

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/342dd05de399b52bb6c54431ff07c1f5167adb0d/test/hooks/AfterSwapPauseBypass.t.sol

## RESOLUTION

Status: Resolved.

MEDIUM M-12

ACKNOWLEDGED

# Bypass of LP Whitelist via EIP-7702

| | |
|---|---|
| CATEGORY | Access Control |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/main/src/hooks/AMMStandardHook.sol#L513 |
| REVIEW | Main Review |

## DESCRIPTION

The `_lpWhitelists` mapping enforces a whitelist of approved liquidity providers, checked in the `addLiquidity` hook. However, with EIP-7702, this restriction can be bypassed.

EIP-7702 allows EOAs to delegate their execution capabilities to smart contracts. For example, if Bob is an approved LP, he can delegate his EOA to a simple proxy contract that exposes an addLiquidity call into LBAMM:

```
contract BobEOA {
address LBAMM;
function addLiquidity() external {
LBAMM.addLiquidity(…);
}
}
```

This enables anyone to call through Bob's proxy, effectively bypassing the whitelist and undermining its purpose.

## RECOMMENDATION

Monitor whitelisted EOAs and consider removing them if they delegate to proxy contracts. Stay up to date with EIP-7702 developments, particularly proposals for delegate runtime introspection (discussion link) [https://ethereum-magicians.org/t/transient-eip-7702-delegate-runtime-introspection-enabling-safer-smarter-wallet-innovation/24050], which may allow safer detection and handling of such cases.

## RESOLUTION

Status: Acknowledged.

MEDIUM M-13

ACKNOWLEDGED

# Pricing Bounds Logic Allows Swaps Outside Limits

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/main/src/hooks/AMMStandardHook.sol#L640 |
| REVIEW | Main Review |

## DESCRIPTION

The `_validatePricingBounds` function is called to enforce min/max price limits. However, the current logic only reverts trades that push the price further out of bounds, while still allowing trades that move the price back into bounds:

- Price below min → only block swaps that push price further down (`zeroForOne`)
- Price above max → only block swaps that push price further up (`!zeroForOne`)

Impact:
- Arbitrage: Users can buy at below-min prices and later sell when price recovers, exploiting the leniency.
- Weaker guarantees for hooks: Token hook creators cannot rely on bounds being enforced strictly, reducing the safety of strategies built on them.
- Oracle-driven pools: In `SingleProviderPoolType`, where price comes from an external oracle, swaps may not move price in the "expected" direction, defeating the intent of the current directional logic.

## RECOMMENDATION

If bounds are meant as hard limits, all swaps should revert when price is outside the configured range.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-14** ACKNOWLEDGED

# LPs Fee undercharged on exact output

| | |
|---|---|
| CATEGORY | Math |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/f14e781f817b099c92493bbd90291e422e836f2a/src/modules/AMMModule.sol#L1361 |
| REVIEW | Main Review |

## DESCRIPTION

In exact output swap, when we ask the AMM to give an exact amount of the output token, the amm figures out how much input it needs to take

but In `_poolExactOutputSwap`

The fee is chosen on a smaller amount than the amount actually traded

```
// before-swap hooks
_executeBeforeSwapHooks();
// @audit, compute pool fee using pre-fee amountOut
uint16 poolFeeBPS = _getPoolFee(swapCache, swapCache.amountOut, swapHooksExtraData);
// then add output hook fees (this increases amountOut)
_applyExactOutputOutputFees(swapCache, tokenInSettings, tokenOutSettings);
// execute pool swap using the now larger amountOut but the earlier lower fee
exactOutputSwap( . , swapCache.amountOut, poolFeeBPS, . );
```

For exact-output we should apply output fees before pool calculation, but the code does it in the opposite order, therefore

The fee now undercharges LPs

The fee is computed based on, Pre-fee amountOut ( what the user wants to receive )

When it should be based on, Post-fee amountOut ( what the pool must actually output )

Because the pool actually executed a larger post-fee amount that under-charges LPs relative to the true swap size

## RECOMMENDATION

we should apply output fees first, then compute the pool fee on the final amount

## M–14 LPs Fee undercharged on exact output CONTINUED

### RESOLUTION

Status: Acknowledged.

MEDIUM **M–15** RESOLVED

# Fee growth miscalculation triggers underflow

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | Dynamic pool |
| REVIEW | Main Review |

## DESCRIPTION

In both dynamic pool swap functions the code seed the swap state with the stored tick

Then, `DynamicHelper.computeSwap` advances through ticks. When the current price lands exactly on a tick boundary, the code set the new current tick

```
// computeSwap()
if (swapCache.sqrtPriceCurrentX96 == step.sqrtPriceNextX96) {
if (step.initialized) { // cross tick, update liquidity }
swapCache.tick = swapCache.zeroForOne ? step.tickNext - 1 : step.tickNext;
}
```

But In both `exactInputSwap` and `exactOutputSwap`, the limit checks permit equality with the hard bounds

```
// exact input
if (zeroForOne) {
// allows sqrtPriceLimitX96 == MIN_SQRT_RATIO
if (sqrtPriceLimitX96 > sqrtPriceCurrentX96 || sqrtPriceLimitX96 < MIN_SQRT_RATIO) revert;
} else {
// allows sqrtPriceLimitX96 == MAX_SQRT_RATIO
if (sqrtPriceLimitX96 < sqrtPriceCurrentX96 || sqrtPriceLimitX96 > MAX_SQRT_RATIO) revert;
}
```

If a user sets `swapExtraData = abi.encode(uint160(MIN_SQRT_RATIO))` for a zero for one swap, the loop can reach exactly `MIN_SQRT_RATIO` with `step.tickNext == MIN_TICK`

At that moment the code sets

```
swapCache.tick = step.tickNext - 1; // MIN_TICK - 1 ( which is out of range )
```

After the swap finishes the code persist that invalid tick

## M–15 Fee growth miscalculation triggers underflow CONTINUED

So the stored state becomes

`sqrtPriceX96 == MIN_SQRT_RATIO`

`tick == MIN_TICK - 1` ( below the supported range )

It could cause a fee accounting underflow because

`_getFeeGrowthInside()` uses

```
if (tickCurrent < tickLower) { inside = lower.outside - upper.outside; }
else if (tickCurrent >= tickUpper) { inside = upper.outside - lower.outside; }
else { inside = global - lower.outside - upper.outside; }
```

With `tickCurrent = MIN_TICK - 1`

For any valid position `tickLower >= MIN_TICK` the first branch is taken

If a position previously recorded `feeGrowthInsideLastX128` while `tick` was valid ( price was near the min ), switching to this below range branch can make the new `feeGrowthInside` smaller than the last value.

The code then does

```
unchecked {
delta0 = feeInside0 - feeInside0Last; // underflows to huge value
}
tokensOwed0 = uint128(FullMath.mulDiv(delta0, position.liquidity, Q128));
```

That unchecked underflow produces an enormous `delta`, which then mints arbitrarily large `tokensOwed` on `collectFees`. The AMM will attempt to pay those fees

If that amount has not occurred in the fee pot yet, the amm will revert

### RECOMMENDATION

Uniswap does not allow sqrtPriceLimit == MIN/MAX sqrtRatio https://github.com/Uniswap/v3-core/blob/main/contracts/UniswapV3Pool.sol#L608-L612

PROOF OF CONCEPT https://github.com/GuardianOrg/amm-pool-type-dynamic-team1-1757356782140/blob/341e62311399e9eda21c364a8f63103a704aef83/test/DynamicPool_fee_underflow.t.sol

### RESOLUTION

Status: Resolved.

MEDIUM **M-16** RESOLVED

# Depth wiped during rounding causes underflow

| | |
|---|---|
| CATEGORY | DoS |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/d41f3a822992a6b5c5cb34291646cbb71dcec2f6/src/libraries/FixedHelper.sol#L253 |
| REVIEW | Main Review |

## DESCRIPTION

In `_calculateLiquidityStartAndEndHeights` we first add the partial in-range depth to `add0` , `add1` , and then round the entire amount down to the pool's height precision

```
// 0-side
add0 += depth0; // include in-range portion
uint256 precisionAddLoss0 = add0 % precision0;
if (precisionAddLoss0 != 0) add0 -= precisionAddLoss0; // @audit, rounds away depth too
liquidityCache.endHeight0 = liquidityCache.startHeight0 + add0;
if (addInRange0) {
liquidityCache.amountAddedOf0To0 = liquidityCache.endHeight0 - currentHeight0; // @audit, can underflow
}
```

The same pattern exists for the 1-side.

This is wrong because, when `addInRangeX == true` , the depth portion ( the remainder between the current height and the previous multiple of `precisionX` ) is already paid for using the other token ( `amountAddedOf1To0` or `amountAddedOf0To1` )

That part must never be rounded away. Because by rounding the entire `addX` after adding `depthX` , we can erase the just-purchased portion and set `endHeightX <= currentHeightX`

Then:

```
liquidityCache.amountAddedOfXToX = endHeightX - currentHeightX;
```

underflows and reverts

Valid in-range only adds or redeposits during withdraw can revert unexpectedly when spacing ≠ 0 and the current height is not aligned because `amountAddedOfXToX` underflows.

if both `addInRange0` , `addInRange1` true, users can be locked out of adding or removing liquidity at certain heights/precisions, creating a DoS on position management.

## M–16 Depth wiped during rounding causes underflow CONTINUED

### RECOMMENDATION

The ideal solution would round only the portion above the depth, not the in-range depth itself. However we also have to be sure to not allow positions that would start and end before the currentHeight. Where depth is the portion up to the currentHeight being paid in the opposite token, keeping the depth as is and only rounding the additional amount that is added on top of the depth ought to preserve this invariant while avoiding underflow.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/e2cafbb8a49a775168e5aa79c3898211570cd05f/test/FixedPoolInRangeDepthRoundingRevert.t.sol

### RESOLUTION

Status: Resolved.

MEDIUM M-17

RESOLVED

# Dust orders possible via scale overflow

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/515f93daf21572eeb96b5be73b3226890c656a7f/src/handlers/clob/CLOBTransferHandler.sol#L160 |
| REVIEW | Main Review |

## DESCRIPTION

The CLOB's group minimum order is calculated as `base × 10^scale` using 256-bit arithmetic

```
minimumOrder := mul(
and(shr(8, groupKey), 0xFFFF), // base (uint16)
exp(10, and(groupKey, 0xFF)) // 10^scale (uint8)
)
```

Because this multiplication is done modulo $2^{56}$

certain ( base, scale ) pairs overflow and wrap to zero

For example, with `scale = 255` we have `10^255 ≡ 2^255 (mod 2^256)`

If `base` is even, then `base × 2^255` is a multiple of $2^{56}$ and evaluates to 0

That happens because the EVM uses 256-bit wraparound, math is done mod $2^{56}$ like a clock

If a product lands exactly on a multiple of $2^{56}$, it's stored as 0

`10^255` and `2^255` differ by a multiple of `2^256`, so they count as the same here.

That sets the group's minimum order to zero

## RECOMMENDATION

We can add a cap where `base × 10^scale` never overflows for any uint16 base, a safe cap is 72 because `65535 × 10^72 < 2^256` but `10^73` risks overflow

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/6c9e1d5a0e3c8ea8e63f406a0a2d37674435a82d/test/handlers/clob/ClobTransferHandler_MinOrderOverflow.t.sol

## RESOLUTION

Status: Resolved.

MEDIUM **M-18** ACKNOWLEDGED

# Flashloan Fee Bypass

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/main/src/modules/AMMModule.sol#L3092 |
| REVIEW | Main Review |

## DESCRIPTION

In `_flashLoan`, once a token hook returns a non-zero `tokenFeeAmount`, the base protocol fee switches from `loanAmount * flashLoanBPS` to `tokenFeeAmount * flashLoanBPS`, and the borrower only repays `tokenFeeAmount + ceil(tokenFeeAmount * flashLoanBPS / MAX_BPS)`.

A hook can set `tokenFeeAmount` to 1 wei, letting users borrow any size loan while paying ~2 wei in fees, so protocol revenue vanishes and flashloans become effectively free.

## RECOMMENDATION

Always charge the base fee on `loanAmount`, independent of hook output, e.g.

```
uint256 base = FullMath.mulDivRoundingUp(flashloanRequest.loanAmount, flashLoanBPS, MAX_BPS);
uint256 feeAmount = base + tokenFeeAmount;
```

Otherwise, add the base fee on top of any hook-managed fee before applying the protocol's share

## RESOLUTION

Status: Acknowledged.

LOW **L-01** RESOLVED

# transferFrom Expecting A Boolean Return Is Used

| | |
|---|---|
| CATEGORY | Warning |
| LOCATION | `AMMModule.sol` |
| REVIEW | Main Review |

## DESCRIPTION

In the `_finalizeSwapCollectFundsAndDisburse` and `_directSwap` functions a `IERC20.transferFrom` call is made using the IERC20 interface which expects a boolean return value.

However, some tokens don't return a boolean and therefore will not work with the raw transferFrom which expects to be able to parse a returned boolean value, leading to a revert.

## RECOMMENDATION

Consider using a low level call which will allow the return value to be zero so long as the tokenIn has code at it's address.

## RESOLUTION

Status: Resolved.

LOW L-02

RESOLVED

# Unexpected Partial Fill Reverts

| | |
|---|---|
| CATEGORY | Error |
| LOCATION | AMMModule.sol |
| REVIEW | Main Review |

## DESCRIPTION

During `exactOut` swaps there is a case where the `swapCache.adjustedAmountSpecified -= amountOutAdjustment` may unexpectedly revert.

This occurs when the before hooks levy a fee increasing the `swapCache.amountOut` from the original value which was stored in the `adjustedAmountSpecified`. Now it's possible for the distance between `swapCache.amountOut` and `actualAmountOut` to exceed the magnitude of the `adjustedAmountSpecified` and cause an underflow panic revert.

Consider the following example:

- User A makes an ExactOut swap with 10 as an amountOut
- The hook fee adds 5 to the amountOut, making it 15
- A partial fill occurs, only filling 3 units of the swap
- amountOutAdjustment = 15 - 3 = 12
- adjustedAmountSpecified is only 10, and therefore panic underflows when being updated

This prevents partial fills from occurring in this edge case and may be unexpected for integrators and users.

## RECOMMENDATION

Consider if this underflow revert case is acceptable. This behavior is a part of a broader issue where fees are applied assuming that the entire swap amount is filled, and may be resolved by scaling the fees according to the fill ratio.

Alternatively, if this revert case should be allowed, consider using an explicit revert to avoid uninformative panic reverts.

## RESOLUTION

Status: Resolved.

LOW L-03

RESOLVED

# WETH Input And msg.value Blocked in directSwap

| | |
|---|---|
| CATEGORY | Unexpected Behavior |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/main/src/modules/AMMModule.sol#L1704, https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/main/src/modules/AMMModule.sol#L2015 |
| REVIEW | Main Review |

## DESCRIPTION

During a `directSwap`, if the executor is required to provide both input and output token, the swap would fail if the input token is wrapped native and the executor sends `msg.value`.

The revert would occur in `_directSwap` as it always expects output token to be wrapped native is `msg.value` is sent:

```
if (msg.value > 0) {
if (swapOrder.tokenOut != address(wrappedNative)) {
revert LBAMM__InputNotWrappedNative();
}
_depositWrappedNativeAndRefundExcess(swapCache.context.executor, directSwapExecutorInput);
swapCache.msgValueUsed = true;
}
```

## RECOMMENDATION

Handle the case where `msg.value` is sent but the input token instead of the output token is wrapped native.

## RESOLUTION

Status: Resolved.

LOW **L-04** RESOLVED

# Single Sided Liquidity Griefing

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L148 |
| REVIEW | Main Review |

## DESCRIPTION

When adding liquidity, the `startHeightX` will use the normalized `currentHeightX` amount (rounded down by `precisionX`), and `endHeightX` will be a multiple of `precisionX` based on the `addX` value.

However, when adding single sided liquidity, with `amountX = 0` but `currentHeightX` not a multiple of `precisionX`, the normalization will likely cause `amountAddedX < valueX` and revert with `FixedPool__LiquidityAddInsufficientForPrecision`

This suggests that the only way to add single sided liquidity will be at pool deployment or if the `amountX = 0` side's `currentHeightX` is exactly at a precision multiple.

In order to avoid the `FixedPool__LiquidityAddInsufficientForPrecision` revert, user will need to provide at least `precisionAddLossX` amount in the opposite side.

Although this might seem as expected behavior, users might be unaware of this and constantly be DoS'ed if any swaps are executed before the liquidity addition.

## RECOMMENDATION

Add a public view preview function that calculates the min amount of the opposite token that they need to deposit in order to prevent single sided liquidity to revert.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/guardian/test/guardian/pocs/SingleSidedLiqDoS.t.sol

## RESOLUTION

Status: Resolved.

LOW L-05

ACKNOWLEDGED

# Height Spacing Cap Not Scaled By Token Decimals

| | |
|---|---|
| CATEGORY | Warning |
| LOCATION | FixedPoolType.sol |
| REVIEW | Main Review |

## DESCRIPTION

In `FixedPoolType.createPool`, both `spacing0` and `spacing1` are capped by a constant:

```
/// @dev Height spacing is capped at a maximum of 24 to avoid excessive height jumps before liquidity becomes active.
uint8 constant MAX_HEIGHT_SPACING = 24;
```

Since spacing is interpreted in raw token units, the cap has very different effects depending on decimals. For 18-decimal tokens this corresponds to ~1e6 units (reasonable), but for 6-decimal tokens it corresponds to ~1e18 units.

As a result, pools with low-decimal tokens can be initialized with overly coarse spacing, causing liquidity to skip large ranges or always round to zero, preventing height advancement.

On the other hand, if precision is set too fine, tiny positions can spam the pool and DoS swaps by forcing excessive height crossings.

## RECOMMENDATION

Consider adjusting max spacing dynamically based on token decimals and prevent extreme values by setting both a minimum and maximum allowed spacing.

## RESOLUTION

Status: Acknowledged.

LOW **L-06** RESOLVED

# removeLiquidity Griefing Via token0/token1 Caps

| | |
|---|---|
| CATEGORY | DoS |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L50 |
| REVIEW | Main Review |

## DESCRIPTION

When users withdraw liquidity from Fixed Pools, they must pass a `FixedLiquidityModificationParams` struct with `amount0` and `amount1`, which represent the maximum amounts of token0/1 to remove.

If the actual withdrawable amounts exceed these caps, the excess is re-deposited as liquidity. This design creates a grief vector:

- It is hard for users to determine the exact `amount0/amount1` values to pass, especially when they want to fully withdraw.
- Even if a user correctly estimates, a swap occurring just before their transaction can change reserves, making their parameters stale.
- The result is a revert with `FixedPool__InsufficientLiquidityForRemoval`, forcing the user to retry and exposing them to DoS griefing.

## RECOMMENDATION

1. Add a public view function (e.g. `previewWithdraw`) that mirrors `_collectPosition` logic without mutating state, returning the exact principal + fees withdrawable.

Alternatively, provide a `withdrawAll` function that removes all liquidity without redepositing excess.

1. Modify withdraw logic to include a slippage guard: let users specify minimum amounts of `token0/token1` to receive.

## RESOLUTION

Status: Resolved.

LOW **L-07** RESOLVED

# computeRatioX96 Returns Inverted Extreme Prices

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/amm-pool-type-dynamic-team1-1757356782140/blob/main/src/libraries/SqrtPriceMath.sol#L241 |
| REVIEW | Main Review |

## DESCRIPTION

When amount1 == 0 or amount0 == 0, the function returns the opposite extreme sqrt price constants. For `sqrt(amount1/amount0)`, if `amount1 == 0` the ratio should be 0 (clamped to `MIN_SQRT_RATIO`), but the code returns `MAX_SQRT_RATIO`; if `amount0 == 0` the ratio is infinite (clamped to `MAX_SQRT_RATIO`), but the code returns `MIN_SQRT_RATIO`. This inverts price direction in edge cases and can lead to severely incorrect pricing, ticks, or pool state if used to derive prices from token amounts.

## RECOMMENDATION

Swap the return values in the zero-amount branches to align with sqrt(amount1/amount0):

```
if (amount1 == 0) return MIN_SQRT_RATIO;
```

```
if (amount0 == 0) return MAX_SQRT_RATIO;
```

## RESOLUTION

Status: Resolved.

LOW **L-08** RESOLVED

# Fee Overflow from Unsafe Downcast

| | |
|---|---|
| CATEGORY | Warning |
| LOCATION | https://github.com/GuardianOrg/amm-pool-type-dynamic-team1-1757356782140/blob/main/src/libraries/DynamicHelper.sol#L336 |
| REVIEW | Main Review |

## DESCRIPTION

During Dynamic Pool swap execution, protocol fees are computed and accumulated as follows:

```
if (swapCache.protocolFeeBPS > 0) {
uint256 delta = FullMath.mulDivRoundingUp(step.feeAmount, swapCache.protocolFeeBPS, MAX_BPS);
step.feeAmount -= delta;
swapCache.protocolFee += uint128(delta);
}
```

The issue arises from downcasting `uint256 delta` uint128 before adding it to `swapCache.protocolFee`.

If `delta > type(uint128).max`, the value silently wraps, leading to under-accounting of protocol fees while the full fee is still deducted from the user's swap flow.

This pattern likely originated from Uniswap v3, where protocol fees were stored as `uint128`. However, in LBAMM, `swapCache.protocolFee` is a `uint256`, making the cast unnecessary.

While practically unlikely under normal conditions, large-fee tokens or extreme-decimal configurations could trigger this overflow, resulting in inconsistent protocol fee tracking and accounting discrepancies.

## RECOMMENDATION

Keep `delta` as a `uint256` to preserve full precision and eliminate the wrapping risk.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-01

RESOLVED

# Token Configuration Access Control Suggestion

| | |
|---|---|
| CATEGORY | Informational |
| LOCATION | `LibOwnership.sol: 35` |
| REVIEW | Main Review |

## DESCRIPTION

In the requireCallerIsTokenOrContractOwnerOrAdmin function the caller is able to show authentication for token configuration on the LimitBreak AMM by either being the contract owner or holding the DEFAULT_ACCESS_CONTROL_ADMIN_ROLE role.

However the DEFAULT_ACCESS_CONTROL_ADMIN_ROLE role can be a dangerous role to perform regular operations with since any address with this permission has the ability to configure all other roles.

With this in mind, it may be a value add for the LimitBreak AMM to also check a designated role which can denote that the holder of this role is specifically authorized to configure this token on the LimitBreak AMM, for those that wish to designate such an address without having the centralization risks of a contract owner or default admin address.

## RECOMMENDATION

Consider introducing a third check which allows those who hold a specific LimitBreak AMM specific role to pass the token configuration authentication.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

# Misleading Documentation

| | |
|---|---|
| CATEGORY | Documentation |
| LOCATION | DynamicPoolType.sol: 269 |
| REVIEW | Main Review |

## DESCRIPTION

In the documentation for the `addLiquidity` function it is mentioned that "A DynamicPoolLiquidityAdded event is emitted with the pool ID, position ID, deposit amounts, and fees".

However the `DynamicPoolLiquidityAdded` that is emitted does not contain the deposit amounts and fees associated with the `addLiquidity` action.

## RECOMMENDATION

Consider adding the deposit0, deposit1, fees0, and fees1 values to the `DynamicPoolLiquidityAdded` event emission.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

ACKNOWLEDGED

# Unexpected ExchangeFee Application

| | |
|---|---|
| CATEGORY | Warning |
| LOCATION | FeeHelper.sol: 70 |
| REVIEW | Main Review |

## DESCRIPTION

In the `calculateAmountAfterFeesExactInput` function the documentation suggests that the fee on top is applied before the exchange fee is applied, which would indicate that the `exchangeFee` basis points apply to the remaining amount in after the fee on top is taken out rather than the entire original amount in.

However the `_calculateBPSFeeWithRecipientAndTaxExactInput` function is invoked using the original `swapCache.amountIn` value which does not factor in the `feeOnTop`. In edge cases this may lead to unexpected panic reverts when the combination of feeOnTop and exchangeFee results in more fees being taken than can be supported.

For example:

- User A uses an exactInput swap with 10 tokens
- The feeOnTop is 5 tokens
- The exchangeFee is 60% based on the original input amount, amounting to 6 tokens
- The total fee is computed as 11 tokens causing a panic revert

## RECOMMENDATION

Consider if the exchangeFee should be applied to the remaining amount input after the feeOnTop is taken.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04

RESOLVED

# Lacking Division By Zero Validation

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | FeeHelper.sol: 219 |
| REVIEW | Main Review |

## DESCRIPTION

For exact-output swaps, the function computes fee = input * BPS / (MAX_BPS - BPS). If BPS equals MAX_BPS (100%), the denominator becomes zero, causing a division by zero revert. Current validation only checks BPS > MAX_BPS, so BPS == MAX_BPS passes validation and leads to an ungraceful panic revert.

## RECOMMENDATION

Consider adjusting the validation such that it reverts gracefully when the feeWithRecipient.BPS value is MAX_BPS.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

RESOLVED

# Redundant PoolHook Checks in directSwap

| | |
|---|---|
| CATEGORY | Gas Optimization |
| LOCATION | `LimitBreakAMM.sol:L364, LimitBreakAMM.sol:L373` |
| REVIEW | Main Review |

## DESCRIPTION

Within `directSwap`, a check is performed twice to revert if either `poolHook` or `poolType` has a nonzero length. This duplication increases code size and gas cost without improving safety, since both checks enforce the same condition.

## RECOMMENDATION

Remove the redundant check and retain only a single validation of `poolHook` and `poolType` length.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-06

RESOLVED

# Outdated Trusted Forwarder Comments

| | |
|---|---|
| CATEGORY | Documentation |
| LOCATION | `LimitBreakAMM.sol:L484, AMMModule.sol:L272` |
| REVIEW | Main Review |

## DESCRIPTION

Trusted forwarder support was removed from the implementation after the previous audit. However, developer comments still reference trusted forwarders in the collect fees functions.

## RECOMMENDATION

Update or remove all outdated references to trusted forwarders in the collect fees functions.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-07

RESOLVED

# Inaccurate _executePoolFeeHook Documentation

| | |
|---|---|
| CATEGORY | Documentation |
| LOCATION | AMMModule.sol |
| REVIEW | Main Review |

## DESCRIPTION

In the NatSpec for the `_executePoolFeeHook` function it is documented that the function `Throws when returned fee exceeds maximum basis points`. However this validation is not implemented in the `_executePoolFeeHook` function and is instead implemented in the `_getPoolFee` function which invokes it.

## RECOMMENDATION

Consider removing the `Throws when returned fee exceeds maximum basis points` statement from the `_executePoolFeeHook` function NatSpec.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-08

RESOLVED

# Partial Fills For Exact Swaps May Be Misleading

| | |
|---|---|
| CATEGORY | Warning |
| LOCATION | `AMMModule.sol` |
| REVIEW | Main Review |

## DESCRIPTION

During the `exactIn` and `exactOut` pool swap flows, partial fills are supported which allow the input amount and output amount to be only a portion of what was requested respectively.

This however goes against the general expected behavior of `exactIn` or `exactOut` swaps, where the user expects to trade exactly their input token amount specified or receive exactly the output token amount specified.

This could raise issues for integrators if they unexpectedly interact with a poolType that performs partial fills, or be unexpected for users if they receive a partial fill.

Furthermore, a trader may want to interact with a pool type that performs partial fills, but may want to enforce that their order is fully filled.

## RECOMMENDATION

Consider adding a field to the SwapOrder that allows the trader to specify whether they are willing to accept a partial fill and potentially even to the extent of magnitude difference that they are willing to accept a partial fill.

Otherwise be sure to clearly document this behavior to users and integrators so they can adjust accordingly for pool types which may perform partial fills.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-09

RESOLVED

# Misleading _updateFixedPoolHeights Variables

| | |
|---|---|
| CATEGORY | Documentation |
| LOCATION | `FixedHelper.sol` |
| REVIEW | Main Review |

## DESCRIPTION

In the `_updateFixedPoolHeights` function some variables are incorrectly named which are indicating that they are different units and values than they actually are.

The [amount0FilledByHeight1](https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/d41f3a822992a6b5c5cb34291646cbb71dcec2f6/src/libraries/FixedHelper.sol#L1024) variable is actually an amount1 unit and it is the share of height0, so it should be named `amount1FilledByHeight0`.

And in the case below, the [amount1FilledByHeight0](https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/d41f3a822992a6b5c5cb34291646cbb71dcec2f6/src/libraries/FixedHelper.sol#L1050) variable is actually an amount0 unit that is the share of height1, so it should be named `amount0FilledByHeight1`.

## RECOMMENDATION

Correct the variable names to `amount1FilledByHeight0` and `amount0FilledByHeight1` respectively.

## RESOLUTION

Status: Resolved.

INFORMATIONAL **I-10** RESOLVED

# Zero Amount Actions

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L966 |
| REVIEW | Main Review |

## DESCRIPTION

In case there is a swap with a very small `amountIn`, `_calculateExactInputLPAndProtocolFee` may result in `amountInAfterFees=0` as pool fees are rounded up. Consequently, both `amount0` and `amount1` can be zero in `_applySwapToLiquidity`.

Similarly, when adding liquidity, there could be cases when both `deposit0` and `deposit1` are zero for a new position, due to precision rounding.

Although there is no direct impact detected, allowing these swaps may result in unexpected behavior.

## RECOMMENDATION

Consider reverting if both `amount0` and `amount1` are zero in `_applySwapToLiquidity`. Additionally, do not allow zero liquidity deposits.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-11

RESOLVED

# Returned Boolean Parameter Not Used

| | |
|---|---|
| CATEGORY | Superfluous Code |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/main/src/libraries/FixedHelper.sol#L651 |
| REVIEW | Main Review |

## DESCRIPTION

The `_addLiquidityToHeight` function returns the boolean `flipped` value if we are adding liquidity to an empty height.

However, this returned parameter is never used in the `_addLiquidity`, which is the only part of the code that uses it.

## RECOMMENDATION

Remove the unused `flipped` return parameter.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-12

RESOLVED

# Incorrect Natspec For collectFees

| | |
|---|---|
| CATEGORY | Documentation |
| LOCATION | https://github.com/GuardianOrg/amm-pool-type-dynamic-team1-1757356782140/blob/main/src/DynamicPoolType.sol#L137 |
| REVIEW | Main Review |

## DESCRIPTION

The `DynamicPoolType.collectFees` function natspec comment states:

```
* @dev Throws when ticks are not aligned with tick spacing.
```

However, this is never enforced as the fee-collection path does not call `_flipTick`. Since invalid positions cannot accumulate tokens or fees, tick alignment is not relevant in this context.

## RECOMMENDATION

Remove or update the natspec comment in `DynamicPoolType.collectFees` to accurately reflect the function's behavior.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-13

RESOLVED

# Underflow In computeRatioX96 For Large amount1

CATEGORY Logical Error

LOCATION https://github.com/GuardianOrg/amm-pool-type-dynamic-team1-1757356782140/blob/main/src/libraries/SqrtPriceMath.sol#L260

REVIEW Main Review

## DESCRIPTION

The while loop that finds a safe scaling factor uses the condition `if (maxMultiplier > multiplier) break; else --n;`. If `maxMultiplier == 1` (which occurs for very large amount1), the loop reaches `n == 0`, does not break (1 > 1 is false), and then decrements n causing an underflow revert in Solidity 0.8+. While practically unreachable for real token amounts, this is a correctness bug that can cause unexpected panics given extreme inputs.

## RECOMMENDATION

Use `>=` instead of `>` to ensure termination at `n == 0` and add a guard before decrementing:

if (maxMultiplier >= multiplier) break;

if (n == 0) break; // safety guard

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-14

RESOLVED

# Liquidity Removal Hooks Cannot Charge Fees

| | |
|---|---|
| CATEGORY | Informational |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/f14e781f817b099c92493bbd90291e422e836f2a/src/modules/AMMModule.sol#L552 |
| REVIEW | Main Review |

## DESCRIPTION

When liquidity is removed, the corresponding remove-liquidity hooks are called. However, unlike Uniswap V4's `afterModifyLiquidity` hooks, these hooks cannot return a delta that adjusts the withdrawn amounts.

As a result, hooks in Limit Break lack the ability to apply custom logic on withdrawals. For example, a protocol may want to penalize early liquidity removal by charging a fee, but the current design does not allow this. This limits the expressiveness and usefulness of liquidity hooks.

## RECOMMENDATION

Consider extending the functionality of remove liquidity hooks to support adjustment of the final withdrawn amounts or apply fees during liquidity removal.

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-15

RESOLVED

# Misleading ErrorMsg in _requireCallerIsRegistry

| | |
|---|---|
| CATEGORY | Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/main/src/hooks/AMMStandardHook.sol#L737 |
| REVIEW | Main Review |

## DESCRIPTION

The `_requireCallerIsRegistry` function reverts with `AMMStandardHook__CallerIsNotRegistryOrSelf()`. However, the check only enforces `msg.sender == address(_creatorHookSettingsRegistry)`.

```
function _requireCallerIsRegistry() internal view {
if (!(msg.sender == address(_creatorHookSettingsRegistry))) {
revert AMMStandardHook__CallerIsNotRegistryOrSelf();
}
}
```

The revert message suggests that both registry and self-calls are valid, but in practice no self-calls are made.

## RECOMMENDATION

Update the revert string to `CallerIsNotRegistry` for accuracy, or add a self-check if self-calls were originally intended.

## RESOLUTION

Status: Resolved.

INFORMATIONAL **I-16** ACKNOWLEDGED

# Top only node self linking edge case

| | |
|---|---|
| CATEGORY | Unexpected Behavior |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/d41f3a822992a6b5c5cb34291646cbb71dcec2f6/src/libraries/FixedHelper.sol#L542 |
| REVIEW | Main Review |

## DESCRIPTION

`nextHeightAbove` always must be at or above `currentHeight` , never below it. If there is no higher step, the code does `nextHeightAbove == currentHeight` like a no higher step marker

When removing liquidity, the code removes two list nodes, the start height and the end height for that position

```
// _removeLiquidityFromHeight()
} else {
mapBelow.nextHeightAbove = nextHeightAbove = nextHeightBelow; // @audit
mapHeight.nextHeightBelow = 0;
mapHeight.nextHeightAbove = 0;
}
newNextHeight = start ? nextHeightBelow : nextHeightAbove;
```

This `a = b = c` chained assignment does two things

Updates the link in storage `mapBelow.nextHeightAbove` and

Mutates the local variable `nextHeightAbove` so it equals `nextHeightBelow`

That local `nextHeightAbove` is later returned as `newNextHeight`, and the caller uses it to set `height.nextHeightAbove`

a top only node self-links `nextHeightAbove == fromHeight`

Its `nextHeightBelow` can be 0 when the node is the only active height, or when it's the lowest height

Because of the chained assignment, we change the local `nextHeightAbove` to 0 right before returning it

The caller then does `height.nextHeightAbove = 0` because we said newNextHeight is 0

## I–16 Top only node self linking edge case CONTINUED

So we end up with `nextHeightAbove = 0` while `currentHeight > 0`, which breaks the invariant ( next above must be > current )

Later, when the pool tries to increase height, it computes

`(nextHeightAbove - currentHeight) * liquidity -`

With `nextHeightAbove < currentHeight`, that underflows and swaps revert

after withdrawing that top/only height, any `_increaseHeight` call will revert

This can triggers when removing the last liquidity at the top endHeight (the top node becomes empty). However this case is only temporary until another user adds liquidity above the current height.

### RECOMMENDATION

Be aware of this edge case where `nextHeightAbove == 0` while `currentHeight > 0` may arise. If no further impacts are identified and this edge case is acceptable, consider using an explicit revert rather than allowing underflow.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/minato7namikazi-patch-5/test/FixedEndHeightRemovalUnderflow.t.sol

### RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-17

ACKNOWLEDGED

# Equal division allows domination of the pot

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | `Fixed pool` |
| REVIEW | Main Review |

## DESCRIPTION

Adding liquidity counts positions, not size

When a new position range includes the current height, the pool increments the liquidity counter by 1 ( not by any amount weighted metric )

```
if (currentHeight >= startHeight && currentHeight < endHeight) {
++height.liquidity; // counts 1 per position
++height.remainingAtHeight; // same
}
```

Here, `height.liquidity` represents the number of overlapping positions, not how much each position adds

Fees are divided equally per position at the height because

During swaps, fees allocated to the current height are divided by `heightCache.liquidity` ( the count above )

```
uint256 feeGrowthGlobalIncrement = UnsafeMath.simpleMulDiv(
feeDistributedToHeight,
Q128,
heightCache.liquidity // divides by count of positions
);
```

Positions claim fees without any weight for size, when claiming, each position just applies the fee growth delta ( no multiplication by the position size )

```
fee0 = (feeGrowthInside0X128 - feeGrowthInside0LastX128) / Q128;
fee1 = (feeGrowthInside1X128 - feeGrowthInside1LastX128) / Q128;
```

Suppose a victim supplies all capital via 1 position ( range covers current height )

Attacker supplies a tiny amount per position but creates N positions ( same height range )

At any height where both are active

## I–17 Equal division allows domination of the pot CONTINUED

height.liquidity = 1 (victim) + N (attacker)`

Each position gets `1 / (N + 1)` of the fees at that height

Attacker can gets `N / (N + 1)` of all fees, with only dust capital per position

As `N` grows, attacker fee share can drain a much larger amount of the pot

### RECOMMENDATION

Not recorded.

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/3b05a176c05c7372b9b057910f4d2da3fccb93ac/test/FixedFeePerPositionSybil.t.sol

### RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-18

ACKNOWLEDGED

# addInRange False Can Still Produce Fill

| | |
|---|---|
| CATEGORY | Unexpected Behavior |
| LOCATION | `FixedHelper.sol` |
| REVIEW | Main Review |

## DESCRIPTION

When providing liquidity in the fixed pool, users have the option to specify an `addInRange` value for each side to opt in or out of deploying liquidity inside an active height precision multiple.

When `addInRange` is false there are no cases where the user's add0/1 is deducted to provide liquidity in the opposite height, however there is a case where a user provides liquidity that is technically partially filled immediately upon creation.

This is because upon removal of an LP position, if there is any remainder at the current height such that `remainingAtHeight != liquidity`, then the removal of the LP position takes 1 unit of the remainder with them as a filled quantity.

When the `currentHeight` is an exact multiple of the precision, the position's `startHeight` is exactly the `currentHeight`, ignoring the `addInRange` value. As a result this remainder fill case applies to `addInRange == false` liquidity additions when the `currentHeight` is on a precision multiple.

This may simply be unexpected for users who specify `addInRange == false` and may break assumptions in integrating systems.

## RECOMMENDATION

Be sure to document this case for integrators of the fixed pools. Otherwise if `addInRange == false` should instead totally avoid immediate fills, even from remainders, then consider moving the `startHeight` back by a full precision multiple in the `currentHeight % precision == 0` case when `addInRange == false`.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-19

ACKNOWLEDGED

# Dynamic-fee pool creation improperly blocked

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/515f93daf21572eeb96b5be73b3226890c656a7f/src/hooks/AMMStandardHook.sol#L607-L616 |
| REVIEW | Main Review |

## DESCRIPTION

Dynamic-fee pools using the `DYNAMIC_POOL_FEE_BPS` value which is `55_555` are not handled in pool-creation validation in the standard hook.

The token-level hook enforces min/max fee bounds against the sentinel value, making dynamic-fee pools fail validation.

In the AMMStandardHook, in `_enforcePoolCreationSettings` function

```
if (tokenSettings.minFeeAmount > 0) {
if (details.fee < tokenSettings.minFeeAmount) {
revert AMMStandardHook__PoolFeeTooLow();
}
}
if (tokenSettings.maxFeeAmount > 0) {
if (details.fee > tokenSettings.maxFeeAmount) {
revert AMMStandardHook__PoolFeeTooHigh();
}
}
```

It compares details.fee to minFeeAmount/maxFeeAmount with no exception for the dynamic sentinel.

Amm Core creation allows dynamic fees and requires a pool hook when fee is dynamic:

```
function _createPool() internal returns (bytes32 poolId) {
_requireAddressHasCode(details.token0);
_requireAddressHasCode(details.token1);
if (details.fee > MAX_BPS) {
if (details.fee != DYNAMIC_POOL_FEE_BPS) {
revert LBAMM__PoolFeeMustBeLessThan100Percent();
} else {
if (details.poolHook == address(0)) {
revert LBAMM__InvalidPoolFeeHook();
```

## I–19 Dynamic-fee pool creation improperly blocked CONTINUED

```
}
}
}
}
```

If a token has any standard maxFeeAmount, creating a dynamic-fee pool using `details.fee = DYNAMIC_POOL_FEE_BPS` will revert with `AMMStandardHook_PoolFeeTooHigh()`. This means that the token must assign the `maxFeeAmount` boundary as 55555 and therefore does not allow it to implement reasonable validations on that dynamic fee.

### RECOMMENDATION

Let this finding document the limitation of the standard hook fee validation, where if a token creator wishes to impose fee range validations on a dynamic fee they must implement their own custom hook with modifications to the `AMMStandardHook`.

### RESOLUTION

Status: Acknowledged.

CRITICAL C-01

RESOLVED

# Non Head Order Closing Traps Maker Funds

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | CLOBHelper.sol |
| REVIEW | Remediation Review |

## DESCRIPTION

Inside `closeOrder`, the code zeroes the order `inputAmount` before computing the refund for the case where the order being closed is not the current head order in the price bucket

```
Order storage ptrOrder = ptrOrderBucket.orders[orderNonce];
if (ptrOrder.maker != maker) {
revert CLOBTransferHandler__InvalidMaker();
}
if (ptrOrder.inputAmount == 0) {
revert CLOBTransferHandler__OrderInvalidFilledOrClosed();
}
ptrOrder.inputAmount = 0; // zeroed here
bytes32 orderId = _orderToOrderId(ptrOrder);
bytes32 currentOrderId = ptrOrderBucket.currentOrderId;
if (orderId == currentOrderId) {
} else {
// Non Head Case
unfilledInputAmount = ptrOrder.inputAmount; // @audit , now always 0
}
```

Because `ptrOrder.inputAmount` is set to 0 first

`unfilledInputAmount` becomes 0 for any non head order

The caller later receives this value in closeOrder

```
uint256 unfilledInputAmount = CLOBHelper.closeOrder();
makerTokenBalance[tokenIn][msg.sender] += unfilledInputAmount; // credits 0
```

Any maker who closes an order that isn't currently at the head of its bucket gets no refund, trapping the user funds inside the contract permanently

There is no way for the maker to recover the locked tokens

`makerTokenBalance[tokenIn][maker]` is not credited ( it adds 0 )

## C-01 Non Head Order Closing Traps Maker Funds CONTINUED

The order is gone, can't be filled later or closed again

`withdrawToken` can't withdraw those tokens because the maker internal balance was never credited

Head of bucket closes happen to work because the refund is taken from `ptrOrderBucket.inputAmountRemaining`, not from `ptrOrder.inputAmount`

### RECOMMENDATION

Compute the refund before zeroing `inputAmount` in the non current branch

### RESOLUTION

Status: Resolved.

CRITICAL C-02

RESOLVED

# Pricing Bounds Check Breaks Direct Swaps

| | |
|---|---|
| CATEGORY | DoS |
| LOCATION | https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1757356686103/blob/515f93daf21572eeb96b5be73b3226890c656a7f/src/hooks/AMMStandardHook.sol#L647 |
| REVIEW | Remediation Review |

## DESCRIPTION

Standard Hook assumes a pool based swap even during direct swaps

The AMM calls token hooks for direct no pools swaps

In _directSwap it executes

`_executeBeforeSwapHooks()` `_executeAfterSwapHooks()`

These functions always call _executeSwapHook(), passing a `HookSwapParams` that does not correspond to a pool hop and thus leaves `swapCache.poolId` at its default, 0

Standard Hook unconditionally enforces pricing bounds on every beforeSwap, afterSwap, by reading the current pool price via the pool type interface

```
// AMMStandardHook, beforeSwap / afterSwap
_validatePricingBounds(swapParams, token, pairToken);
// AMMStandardHook, _validatePricingBounds
address poolType = PoolDecoder.getPoolType(params.poolId);
uint160 sqrtPriceX96 =
ILimitBreakAMMPoolType(poolType).getCurrentPriceX96(msg.sender, params.poolId);
```

Because poolId will be 0 during a direct swap,

PoolDecoder.getPoolType(0) becomes zero address, Then that call in the standard hook will revert `ILimitBreakAMMPoolType(address(0)).getCurrentPriceX96()`

any token that enable before/after hook flags and set tokenHook to the Standard Hook, will cause all the token direct swaps to always revert, resulting in a DoS

## RECOMMENDATION

Since DirectSwap is a peer-to-peer trade with no pool price impact, early return if no poolId in `_validatePricingBounds` ,

## C-02 Pricing Bounds Check Breaks Direct Swaps CONTINUED

### RESOLUTION

Status: Resolved.

CRITICAL C-03

RESOLVED

# Height List Corruption Breaks Swaps

| | |
|---|---|
| CATEGORY | DoS |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/202509_Debug_Double_Fee_Claim/src/libraries/FixedHelper.sol#L627-L628, https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/202509_Debug_Double_Fee_Claim/src/libraries/FixedHelper.sol#L741-L752 |
| REVIEW | Remediation Review |

## DESCRIPTION

`_addLiquidityToHeight` relies on the caller's `endHeightInsertionHint` to locate the proper slot in the height linked list. When the hint references a height that is not active, the code takes the fallback branch `(informationNextHeightBelow | informationNextHeightAbove == 0)` and resets informationHeight to 0:

```
if (informationNextHeightBelow | informationNextHeightAbove == 0) {
if (informationHeight == 0) {
if (toHeight != 0) {
mapInformationHeight.nextHeightAbove = toHeight;
mapToHeight.nextHeightAbove = toHeight; // points to itself
}
break;
} else {
informationHeight = 0;
mapInformationHeight = heightMap[informationHeight];
}
}
```

This fallback assumes `heightMap[0]` always has valid neighbor pointers, but `_removeLiquidityFromHeight` clears both pointers (`mapHeight.nextHeightBelow = 0;` `mapHeight.nextHeightAbove = 0;`) whenever a height "flips" to zero gross liquidity—even for height 0. That means the sentinel can be left completely disconnected while other heights remain in the list.

Consequently, if the fallback path runs after height 0 was cleared, the new node is wired directly to the 0 sentinel (and to itself). The linked list is now split and height traversal stalls.

Proof of Concept (see attached link):

1. Active heights: `0 → 260_000 → 152_478_000 → 152_504_000`
2. Liquidity is withdrawn from `(0, 152_478_000)`, so `_removeLiquidityFromHeight` clears

## C-03 Height List Corruption Breaks Swaps CONTINUED

the entry at height 0.
3. Liquidity is re-added over `(260_000, 152_915_000)`. Because the supplied hint `104` is stale, `_addLiquidityToHeight` hits the fallback and rewires the sentinel:

`heightMap[0].nextHeightAbove = 152_915_000`,
`heightMap[152_915_000].nextHeightBelow = 0, nextHeightAbove = 152_915_000`

1. Height `152_504_000` still has `nextHeightAbove = 152_504_000`, so walking "up" from it loops forever. During the subsequent swap the height walk stops at `152_504_000` even though higher nodes exist `152_915_000 and 4.4e21`.

Any time a caller provides a stale insertion hint after height 0 has been removed, the height linked list becomes inconsistent. Swaps then fail to reach higher liquidity, effectively bricking all swaps.

### RECOMMENDATION

The main issue is that in `_removeLiquidityFromHeight` the `mapBelow` for the sentinel height is in fact the sentinel height itself, and so assigning `mapBelow.nextHeightAbove` and then assigning `mapHeight.nextHeightAbove` to zero clears out the `nextHeightAbove` for the `mapBelow`.

Do not clear neighbor pointers for the sentinel height. In `_removeLiquidityFromHeight`, gate the pointer reset so height 0 is preserved:

```
if (fromHeight != 0) {
mapHeight.nextHeightBelow = 0;
mapHeight.nextHeightAbove = 0;
}
```

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-core-fuzz-1757356641758/blob/poc-corrupt-heightMap/test/fuzzing/FoundryPlayground.sol#L34

### RESOLUTION

Status: Resolved.

HIGH **H-01** RESOLVED

# Hook Fee Accounting Error

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/202509_Add_Direct_Output_Validation_PoC/src/modules/AMMModule.sol#L742 |
| REVIEW | Remediation Review |

## DESCRIPTION

Recent liquidity-path changes started storing hook-returned fees (e.g. `hookFee0, hookFee1`) in `_executeTokenLiquidityCollectFeesHook` and `_executeTokenModifyLiquidityHook`.

When token0's hook returns a fee payable in token1, `_storeHookFees` should use token1's settings to decide whether the hook or the token admin owns that balance. Instead, the call still passes token0's settings. Because _storeHookFees derives its storage key from the provided settings, the fee is written into token0's namespace.

As a result, if token0's settings mark "hook manages fees," token0's hook can now withdraw token1's funds via `collectHookFeesByHook`. Conversely, if token1 expects its hook to manage fees but token0's settings don't, the fee ends up under the token-managed bucket and token1's hook can never claim it.

## RECOMMENDATION

Whenever `_storeHookFees(tokenFor, …)` is called, load TokenSettings for tokenFor before the call. E.g. for hookFee1, fetch `Storage.appStorage().tokenSettings[context.token1]` and pass that structure.

PROOF OF CONCEPT https://github.com/GuardianOrg/amm-pool-type-dynamic-team1-1757356782140/blob/main/test/POC_HookFeeAccountingError.t.sol

## RESOLUTION

Status: Resolved.

HIGH **H-02** RESOLVED

# Side Mix Precision Issue In The Quoter

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | Fixed Pool Quoter |
| REVIEW | Remediation Review |

## DESCRIPTION

In the added `processQuoteValueRequiredForInRangeAdd` which compute how much of the paired token is required to add liquidity in range in `FixedPoolQuoter`

The precision for token1 is fetched using the token0 flag

```
function processQuoteValueRequiredForInRangeAdd(bytes32 poolId) external view returns (uint256 amount0, uint256 amount1) {
FixedPoolState storage ptrPoolState = pools[poolId];
// token0 path — Correct
uint256 precision0 = FixedPoolDecoder.getPoolHeightPrecision(poolId, true);
uint256 inRangeDepth0 = ptrPoolState.height0.currentHeight % precision0;
if (inRangeDepth0 != 0) {
amount0 = precision0 - inRangeDepth0;
amount1 = FixedHelper.calculateFixedInput(inRangeDepth0, ptrPoolState.sqrtPriceX96, true);
}
// token1 path — 'true' is used again should be 'false'
uint256 precision1 = FixedPoolDecoder.getPoolHeightPrecision(poolId, true); // @audit, wrong side
uint256 inRangeDepth1 = ptrPoolState.height1.currentHeight % precision1;
if (inRangeDepth1 != 0) {
amount1 += precision1 - inRangeDepth1;
amount0 += FixedHelper.calculateFixedInput(inRangeDepth1, ptrPoolState.sqrtPriceX96, false);
}
}
```

This could lead to massive under / over quoting when both sides spacings differ

`getPoolHeightPrecision(poolId, side0)` should return the side specific precision

## H-02 Side Mix Precision Issue In The Quoter CONTINUED

### RECOMMENDATION

```
- uint256 precision1 = FixedPoolDecoder.getPoolHeightPrecision(poolId, true);
+ uint256 precision1 = FixedPoolDecoder.getPoolHeightPrecision(poolId, false);
```

### RESOLUTION

Status: Resolved.

HIGH **H-03** RESOLVED

# Pool Creators Can Avoid Paying Protocol Fees

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | `AMMModule.sol` |
| REVIEW | Remediation Review |

## DESCRIPTION

In the `_poolSwapByInput` function the `_validateProtocolFees` validation occurs based on the `expectedProtocolLPFee` which is estimated by the `amountIn` and `poolFee`. However this may not be accurate to the actual fee values returned by the `poolType`.

In the event that the `poolType` returns a `poolFeeOfAmountIn` that is larger than the fee that the `expectedProtocolLPFee` was computed based on the `_validateProtocolFees` validation does not require that the additional excess fees had a corresponding increase in the protocol fees.

For example:

- amountIn = 100
- poolFee = 1%
- lpFee = 5%
- Total pool fees predicted is 1 token, of which 0.95 is attributed to the pool and 0.05 is attributed to the protocol
- The pool actually returns a total fee of 2 tokens, and attributes `poolFeeOfAmountIn` as 1.95 and protocolFee of 0.05
- The predicted protocol fee is 0.05 so the _validateProtocolFees validation is satisfied, however the pool got away with paying half of it's lpFee rate to the protocol

This means pool creators can simply create pools in the Core AMM which purport to have a very small poolFee, causing the `expectedProtocolLPFee` to be very small, while actually charging a much higher fee on swaps via the returned `poolFeeOfAmountIn` value.

## RECOMMENDATION

Consider enforcing that the total fee returned by the protocol is in line with the `poolFee` defined in the core AMM. Otherwise consider enforcing that the resulting `protocolFee` is above the minimum threshold based upon the actual returned fee values from the `poolType`.

## H-03 Pool Creators Can Avoid Paying Protocol Fees CONTINUED

### RESOLUTION

Status: Resolved.

MEDIUM M-01

RESOLVED

# Position Hook Used To Avoid Token Ruleset

| | |
|---|---|
| CATEGORY | Gaming |
| LOCATION | `AMMModule.sol` |
| REVIEW | Remediation Review |

## DESCRIPTION

In the `AMMModule` liquidity operation flows, the `_storeNonTokenHookFees` function is used to store hook fees collected by either the position hook or the pool hook. The collection of this fee with the `collectHookFeesByHook` function allows a `recipient` address to be specified who will receive the fees accumulated.

A user can potentially leverage this to avoid a token ruleset which blacklists their address for a token. The user can provide their own liquidity hook for a position, where the liquidity hook specifies the entire remaining fee collection, or liquidity removal amounts as a fee for the position hook itself. This way the funds that would have been attempted to transfer to the user, winding up in the `tokensOwed` mapping entry for the user, are instead redirected to the `tokensOwed` mapping for the `hookFeeKey` which can be redirected to an arbitrary `recipient`.

## RECOMMENDATION

Consider validating that the user would have been able to transfer these tokens to the hook account by the ruleset.

## RESOLUTION

Status: Resolved.

MEDIUM M-02

RESOLVED

# requireExecutorIsPayer Can Be Bypassed

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/f14e781f817b099c92493bbd90291e422e836f2a/src/modules/AMMModule.sol#L2053-L2058 |
| REVIEW | Remediation Review |

## DESCRIPTION

If either token requires executor must be payer, the AMM intends to ensure that the executor provides the input tokens when a custom transfer handler is used and it tries to enforce this by checking the executor balance delta before and after the handler call

That does not prove that the tokens sent to the AMM came from the executor. an executor controlled handler can

1 - pull amountIn tokens from a third party, sending them to the Amm
2 - simultaneously transfer amountIn tokens out of the executor to any controlled place to fake the balance drop

The check passes, but the executor did not actually fund the swap, a third party still can trade

## RECOMMENDATION

we could disallow transfer handlers when the token requires only the executor to pay

## RESOLUTION

Status: Resolved.

MEDIUM M-03

ACKNOWLEDGED

# Partial Fills Have Insufficient MEV Protection

| | |
|---|---|
| CATEGORY | MEV |
| LOCATION | AMMModule.sol |
| REVIEW | Remediation Review |

## DESCRIPTION

The `minAmountSpecified` has been introduced to allow users to specify the extent to which they are willing to accept a partial fill. However in the scenario where a user does allow a partial fill and a partial fill does occur, there user lacks the ability to provide an execution price slippage limit that is accurate to the partiality of the fill.

For example:

- User A would like to swap token A in for 10 token B out using an exactOut swap, the current market price of token A is 1:1 with token B
- User A would accept a minimum execution price of 0.9 token B per token A, so user A specifies a limitAmount of `10/0.9 = 11.1` token A in.
- User A would also accept a partial fill of 80%, or 8 token B output, so User A specifies a `minAmountSpecified` of 8 token B output.
- A partial fill occurs, and user A spends 11 Token A to be filled for 8 Token B, this however breaches User A's desired execution price of 0.9 token B per token A with an actual execution of `8/11 = 0.73` token B per token A.

This lacking ability to protect a user's desired fill price for orders that would accept a partial fill opens such orders up to notable MEV opportunities.

## RECOMMENDATION

Consider if a user should be able to specify an execution price limit whereby on partial fills the user can ensure that the partial fill does not breach a desired execution price.

## RESOLUTION

Status: Acknowledged.

LOW **L-01** ACKNOWLEDGED

# Partial Fill Fee Overcharge

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | `AMMModule.sol: 1369–1371` |
| REVIEW | Remediation Review |

## DESCRIPTION

Partial fills on exact-in swaps still settle hook fees and hop-fee top-ups using the original requested amount. The partial-fill adjustment only scales exchange/protocol fee fields, so token hook-fee, token hop-fee and dynamic pool-fee remain overstated, leading to inflated charges and protocol over-collection.

## RECOMMENDATION

When `actualAmountIn` differs from `originalAmountIn`, proportionally scale every hook-fee field (tokenInTokenInFee, tokenOutTokenInFee, tokenInTokenOutFee, tokenOutTokenOutFee) and the hop-fee protocol accrual to match the filled amount before running settlement.

## RESOLUTION

Status: Acknowledged.

LOW **L-02** ACKNOWLEDGED

# Liquidity positionId Manipulation

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | `AMMModule.sol` |
| REVIEW | Remediation Review |

## DESCRIPTION

In the `_positionCollectFees`, `_positionAddLiquidity`, `_positionRemoveLiquidity` functions the `context.positionId` is declared as a result of the `poolType` interactions. The documentation for the `LiquidityContext.positionId` field purports that it is a `Unique identifier for the liquidity position`.

However, there is no validation performed on the resulting `context.positionId`, and thus any arbitrary `poolType` may spoof a `positionId` that is already in use. This may be misleading for any of the downstream hooks which may rely on the provided `positionId` for accounting.

For example, if a hook used the `positionId` to track how close to a liquidity limit a certain account was, this value could be errantly increased by returning the same `positionId` result from a malicious `poolType`.

## RECOMMENDATION

Consider if there should be validation on the uniqueness of the `positionId`, or if it should be required to correspond to the `ammBasePositionId` in some way. Otherwise, clearly document that hooks should not rely on the `context.positionId` as it is unsafe.

## RESOLUTION

Status: Acknowledged.

LOW **L-03** RESOLVED

# Permit Cosignatures Miss Important Fields

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | `PermitTransferHandler.sol` |
| REVIEW | Remediation Review |

## DESCRIPTION

In the `PermitTransferHandler` the co-signer is intended to expose Web2 co-signing service functionality to include a second layer of signature verification for permit transfers through the handler.

However the digest that is verified with the cosignature does not include several key pieces of data which determine the action being taken on the exchange:

- The `from` address
- The `tokenIn` being used
- The `feeOnTop.recipient`
- The `feeOnTop.amount`

As a result, the cosignature cannot require any intended `feeOnTop`, `from` address to be the payer of the action nor a particular `tokenIn` as the input token for the action. Thus the cosignature could be used for actions which use a different `from` and `tokenIn` as long as they match the same `recipient`, `tokenOut`, `amountSpecified`, `limitAmount`, `exchangeFeeRecipient`, `exchangeFeeBPS`, and `hook`.

## RECOMMENDATION

Consider including these missing values in the cosignature validation, especially the `tokenIn` and `from` addresses.

## RESOLUTION

Status: Resolved.

LOW **L-04** RESOLVED

# Permit Cosignatures Are Not Consumed

| | |
|---|---|
| CATEGORY | Logical Error |
| LOCATION | `PermitTransferHandler.sol` |
| REVIEW | Remediation Review |

## DESCRIPTION

In both the `_executeFillOrKillPermit` and `_executePartialFillPermit` functions, the permit signature nonce/salt is consumed immediately or when the order is fully filled.

However the corresponding cosignature is never invalidated with a nonce when the action's nonce/salt is consumed by the `permitProcessor`.

As a result, the cosignature can be used for multiple duplicate actions by the same executor. This creates a gap in the Web2 signing key service's ability to control the actions that may be relayed onchain, since once a cosignature is produced it technically allows an infinite count of those actions to be carried out before the expiry.

## RECOMMENDATION

Consider introducing and consuming a nonce of the corresponding cosignature immediately in the `_executeFillOrKillPermit` function. Furthermore, in the `_executePartialFillPermit` function, the cosignature could also be immediately consumed when a new order is first created, notice that this would also relax the `cosignatureExpiration`'s affect on the overall order validity, so consider if that is intended and whether it should be maintained.

## RESOLUTION

Status: Resolved.

LOW **L-05** RESOLVED

# Cosigner Cannot Be Destroyed Across Systems

| | |
|---|---|
| CATEGORY | Warning |
| LOCATION | Global |
| REVIEW | Remediation Review |

## DESCRIPTION

In the original `PaymentProcessorV2` implementation, the `destroyCosigner` function verified that the cosigner simply signed the digest: `ECDSA.toEthSignedMessageHash(bytes(COSIGNER_SELF_DESTRUCT_MESSAGE_TO_SIGN))`.

The cosigner logic used in LBAMM is borrowed from the PaymentProcessorV2 system, however there are several key differences with the new cosigner implementation that prevent the `destroyCosigner` action from being compatible across implementations.

Firstly, the digest used in the PermitTransferHandler of the LBAMM is still keyed with a domain separator that uses the name, version, and verifying contract in the domain. The chainId has been removed, but this domain as is still means that the signature must be re-generated if there is ever a PermitTransferHandler instance which does not sit at the same address, using the same name, and the same version on all chains.

Secondly, the `COSIGNER_SELF_DESTRUCT_MESSAGE_TO_SIGN`, defined as `"COSIGNER_SELF_DESTRUCT"` in `PaymentProcessorV2` is different from the new signed contents of the `COSIGNER_SELF_DESTRUCT_TYPEHASH` and cosigner hash. Furthermore, the original PaymentProcessorV2 version uses `toEthSignedMessageHash` which pre-pends `\x19Ethereum Signed Message:\n32` to the resulting hash's pre-image, while the new hashing approach pre-pends the bytes 0x1901 to the pre-image.

This means that if a cosigner were to be used across products, or used by multiple verifying contracts on the same network, then multiple signatures would have to be generated to destroy the cosigner in the entire world.

## RECOMMENDATION

Consider standardizing on one message that ought to be signed by a co-signer across products. Or explicitly document that a co-signer should not be used across products.

Furthermore, consider removing the domain entirely from the `destroyCosigner` digest, in case there ought to be future uses of a co-signer where it may be used in a different validator/transferHandler on the same network.

## L-05 Cosigner Cannot Be Destroyed Across Systems CONTINUED

### RESOLUTION

Status: Resolved.

LOW **L-06** RESOLVED

# Division By Zero Virtual Reserves

| | |
|---|---|
| CATEGORY | Error |
| LOCATION | https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1757356720635/blob/2025209_Debug_Double_Fee_Claim/src/libraries/FixedHelper.sol#L1120 |
| REVIEW | Remediation Review |

## DESCRIPTION

For Fixed pools where one side's reserve is completely empty, `_updateExpectedReserve` leaves `swapCache.expectedReserve` at 0. Subsequently, `_updateFixedPoolHeights` then divides by zero and reverts with `FullMath__MulDivOverflowError`.

## RECOMMENDATION

Consider adding an explicit guard immediately after `_updateExpectedReserve` to sanity-check `swapCache.expectedReserve`. When it is zero, revert with a purpose-built error (e.g. `FixedPool__InsufficientExpectedReserve()`)

## RESOLUTION

Status: Resolved.

LOW **L-07** RESOLVED

# Missing Overflow Check

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | `AMMModule.sol` |
| REVIEW | Remediation Review |

## DESCRIPTION

In the `_applySwapByOutputOutputFees` function the fee accounting logic is wrapped in a try/catch block which carefully performs overflow checks on the swapAmountOut result, however does not implement an overflow check for the `outputProtocolFeeFromHookFees` increment in the `tokenOutTokenOutFee` case.

It may be extremely unlikely or even technically impossible given the relevant constraints for an overflow to occur on the `outputProtocolFeeFromHookFees` variable, however out of an abundance of caution an overflow check should be added.

The same check can also be added to the `protocolFeeFromHookFees` variable addition in the `_applySwapByInputInputFees` function.

## RECOMMENDATION

Consider adding an overflow check for the addition to the `outputProtocolFeeFromHookFees` variable in the `_applySwapByOutputOutputFees` function for the `tokenOutTokenOutFee` fee handling case as well as for the `_applySwapByInputInputFees` variable in the `_applySwapByInputInputFees` function `tokenOutTokenInFee` case.

## RESOLUTION

Status: Resolved.

LOW L-08

ACKNOWLEDGED

# Output Swaps Allow Pools To Underpay

| | |
|---|---|
| CATEGORY | Validation |
| LOCATION | `AMMModule.sol` |
| REVIEW | Remediation Review |

## DESCRIPTION

In the `_poolSwapByOutput` function the `_validateProtocolFees` validation occurs based on the `poolType` provided `poolFeeOfAmountIn` and `swapCache.protocolFee`.

This is in contrast to the validation for `_poolSwapByInput` which validates the minimum fee based on the `expectedProtocolLPFee` which is calculated based on non-poolType controlled variables such as the `amountIn` and the `poolFee`.

Therefore, for exact output swaps, the poolType can return totalFees which are smaller than the core AMM expects and still pass the `_validateProtocolFees` validation.

## RECOMMENDATION

Consider if it is acceptable for arbitrary poolTypes to charge total fees that are less than the protocol expects and as a result pay less protocol fees on output swaps.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01** ACKNOWLEDGED

# Users min/max Protections Can Be Bypassed

| | |
|---|---|
| CATEGORY | Documentation |
| LOCATION | AMMModule.sol |
| REVIEW | Remediation Review |

## DESCRIPTION

In `_positionRemoveLiquidity`

1 ) We ask the pool for the withdrawal/fee

2 ) Check user min/max before hooks

3 ) Calling the code that lets hooks charge fees

4 ) Then apply accounting and send tokens

The same ordering exists for add liquidity in `_positionAddLiquidity` ( min/max checked before hooks ) , but hooks may collect extra with user caps

The min/max parameters are supposed to protect the LP against getting fewer tokens than expected on remove, or paying more than expected on add

a hook can legally take fees that push the final net amounts to violate the user chosen ( maxAmount, minAmount ) yet the transaction still succeeds, example

Pool returns ( deposit0 = 100 , fees0 = 0 )

User sets ( maxAmount0 = 100 , maxHookFee0 = 100 )

Hook returns ( hookFee0 = 100 )

But the Net the user actually pays is

`deposit0 - fees0 + hookFee0 = 200`, above the user maximum that he previously chose ( maxAmount0 = 100 )

## RECOMMENDATION

Document that net amounts may exceed or fall below the min/max limits

PROOF OF CONCEPT https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/4dffc29b3e2e6c8b134203e8262a2240e282ef00/test/MinMaxBypass.t.sol

## I-01 Users min/max Protections Can Be Bypassed CONTINUED

### RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-02** RESOLVED

# Executor Payment Check For directSwap

| | |
|---|---|
| CATEGORY | DoS |
| LOCATION | https://github.com/GuardianOrg/lbamm-core-team1-1757356637441/blob/f14e781f817b099c92493bbd90291e422e836f2a/src/modules/AMMModule.sol#L2053-L2058 |
| REVIEW | Remediation Review |

## DESCRIPTION

The system allows an option for tokens to enforce "executor must be the one who actually pay"

whenever a token sets `TOKEN_SETTINGS_REQUIRE_PAYER_IS_EXECUTOR_FLAG` in the amm token settings

when a custom transfer handler is used, the amm enforces that rule by checking the executor balance of `tokenIn` before and after the handler runs

```
// _finalizeSwapCollectFundsAndDisburse
if (swapCache.requireExecutorIsPayer) {
executorTokenInBalanceRequired =
IERC20(swapOrder.tokenIn).balanceOf(swapCache.context.executor);
if (IERC20(swapOrder.tokenIn).balanceOf(swapCache.context.executor) >
executorTokenInBalanceRequired) {
revert LBAMM__ExecutorDidNotPayInput();
}
}
```

assuming the executor always pays in `tokenIn` for every swap path

That's true for amm pool swaps, both input and output specified, the executor always supplies `tokenIn`

But for direct swaps, the executor/taker supplies `tokenOut`

`_directSwap` collect `swapOrder.tokenOut` from the executor

Then the finalization function pulls maker `tokenIn` while checking for an executor balance drop in `tokenIn`

the executor actually paid earlier in `tokenOut`

The balance will never drop in the check, and we will always revert an honest taker/executor

## I-02 Executor Payment Check For directSwap CONTINUED

This deny DirectSwap P2P taker/maker flow for any token that activated the flag

### RECOMMENDATION

If it is intended to only allow swaps in pools and not peer-to-peer in this option, document it in the codebase and for token owners

### RESOLUTION

Status: Resolved.

§ 05

# Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

# About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

**$45B+**
DIGITAL ASSETS UNDER REVIEW

**200+**
ENGAGEMENTS COMPLETED

**500+**
CRITICAL & HIGH FINDINGS REPORTED

Reach us at audits@guardianaudits.com.